Bachelor's Thesis | 2026

Accelerometer Calibration Using Embedded Neural Networks

Analytical Calibration, Embedded Inference
and Continual Learning with Python and STM32

Francesco De Tursi

MSc student in Mechanical Engineering
University of Bologna

LinkedIn | GitHub | ffrancesco.detursi@gmail.com

Project overview

This bachelor's thesis was completed at the University of Bologna in the academic year 2025/2026, in collaboration with IRMA L@b, under the supervision of Prof. Edoardo Ida' and co-supervision of Matteo Dall'Olio. It investigates accelerometer calibration using compact neural networks. Static measurements from a Murata SCL3300 support analytical calibration and supervised training in Python, followed by deployment on an STM32 microcontroller. A separate study explores physically constrained learning and adaptation to a simulated change in sensor gain and bias.

Abstract

Measurement reliability directly affects control and operational monitoring in mechanical and automated systems, making sensor-error correction necessary. This thesis investigates accelerometer calibration using neural networks executable on a microcontroller, assessing both the reproduction of an analytical correction and learning guided by the gravitational norm constraint.

Experiments used a Murata SCL3300 accelerometer connected to an STM32 Nucleo-H743ZI2 board. Measurements were acquired in different static sensor orientations and used to identify a nine-parameter analytical calibration, whose outputs then served as neural network training targets. Comparison of several architectures led to selection of a 73-parameter network, exported to ONNX and integrated into the firmware through STM32Cube AI Studio. On twelve test poses, the network reduces the RMSE of pose-mean norm deviations by 88.68% relative to Factory measurements, while analytical calibration retains smaller errors. Tests on ten additional poses also confirm correct operation of acquisition and inference directly on the microcontroller.

Finally, an unsupervised network is investigated, trained with the norm constraint without calibrated measurement targets. Starting from this model, a continual learning procedure is simulated in Python by introducing a controlled gain and bias change and assessing how far sequential updates reduce error relative to the initial fixed-weight network.

Contents

Abstract i

1 Introduction 1

2 Accelerometers and Neural Network Calibration 3
- 2.1 Operating Principles of Accelerometers . . . 3
- 2.2 Measurement Errors and Calibration . . . 5
- 2.3 Neural Networks for Calibration . . . 6
- 2.4 Neural Networks on Microcontrollers . . . 7

3 Experimental Setup and Data Acquisition 8
- 3.1 Sensors and the STM32 Nucleo Board . . . 8
- 3.2 Connections and Software Tools . . . 8
- 3.3 Acquisition Procedure . . . 10
- 3.4 Data Organisation . . . 10

4 Analytical Calibration of the Accelerometer 13
- 4.1 Mathematical Model . . . 13
- 4.2 Parameter Identification . . . 14
- 4.3 Validation and Results . . . 16
- 4.4 Directional Verification and Limitations . . . 18

5 Neural Network Design and Training 21
- 5.1 Dataset Preparation . . . 21
- 5.2 Architectures and Training . . . 22
- 5.3 Model Selection . . . 23
- 5.4 Offline Evaluation . . . 25

6 Embedded Implementation and Experimental Verification 28
- 6.1 Export and Firmware Integration . . . 28
- 6.2 Numerical Verification of Deployment . . . 29
- 6.3 Comparison on New Acquisitions . . . 31
- 6.4 Performance Discussion . . . 33

7 Unsupervised Learning and Continual Learning 35
- 7.1 Principles and Applications of Continual Learning . . . 35
- 7.2 Unsupervised Pretraining with the Norm Constraint . . . 36
- 7.3 Evaluation of the Unsupervised Network . . . 38
- 7.4 Simulation of Adaptation to Calibration Drift . . . 39
- 7.5 Adaptation Results and Model Comparison . . . 41

8 Conclusions and Future Work **43**
8.1 Results and Concluding Remarks 43
8.2 Future Work 44

A Pose Acquisition Code **45**

B Analytical Calibration Code **47**

C Neural Network Training Code **49**

D Network Pretraining Code for Continual Learning **52**

E Code for Adaptation to Calibration Drift **54**

References **55**

Chapter 1

Introduction

The evolution of industrial systems and automated machinery has led to increasingly stringent requirements for accuracy, speed, reliability and efficiency. In many fields, from robotics and packaging machinery to handling systems, operations must be performed with high precision and repeatability while maintaining high performance and low costs. Achieving these objectives depends not only on the design of the mechanical structure and actuators, but also on the ability to determine the system's state with sufficient accuracy during operation. Modern machines therefore incorporate numerous sensors to acquire information on quantities such as position, velocity, acceleration, force and temperature. In closed-loop control systems, these measurements provide the feedback used to compare actual and desired behaviour and determine the necessary corrections. The growing volume of sensor information thus gives rise to two closely related needs: sufficiently reliable data and rapid, efficient processing. Measurement quality is consequently crucial, since sensor errors can adversely affect subsequent processing and overall system performance.

Accelerometers are among the most widely used measurement devices in mechatronics, with applications in vibration analysis, motion monitoring and orientation sensing. Like all real sensors, accelerometers exhibit non-ideal behaviour that can cause discrepancies between the true and measured values of a quantity. Calibration identifies and compensates for systematic errors, improving the accuracy of the acquired data. Alongside traditional analytical calibration approaches, the increasing availability of data and advances in computing have encouraged the use of machine learning techniques for measurement processing, including neural networks. These models learn a relationship between inputs and outputs from data by updating their parameters during training, then apply the learned relationship to new data. The ability to develop compact models and run them directly on microcontrollers makes this approach particularly attractive for embedded systems, both because these devices have limited computational and memory resources and because local processing, on the same device that acquires the measurements, avoids transferring data to external computing systems.

Within this context, this thesis investigates neural networks for accelerometer calibration through two distinct approaches. In the first, data collected under static conditions are used to derive an analytical calibration and obtain corrected sensor measurements. These measurements then provide the reference for training a neural network to learn the same input–output transformation. After training and offline verification, the network is implemented on an STM32 Nucleo-H743ZI2 board, where measurements are corrected directly on the microcontroller. The ultimate objective is to establish whether a sufficiently compact model can reproduce this correction and be transferred to an embedded system. The architecture comparison uses the same number of samples per pose, while the selected model is evaluated against the analytical solution both offline

and through new experimental tests on the microcontroller. The second approach follows this first part of the work and uses a separate network, trained using the physical constraint that, under static conditions, the acceleration norm measured by an ideal sensor equals gravitational acceleration. Training therefore uses the deviation from this value to determine the network parameters, without taking analytically calibrated measurements as targets. Starting from the resulting weights, a continual learning approach subsequently updates the model using measurements modified by a controlled change in gain and bias, while retaining the norm constraint as the learning reference. This approach investigates whether the correction can progressively adapt to a simulated sensor change, comparing the updated model with the unsupervised network whose weights remain unchanged. The method is developed and simulated in Python; implementation and validation on the embedded platform are left for future work.

The thesis comprises eight chapters. Following this introduction, Chapter 2 presents accelerometers and the principles underlying their calibration using neural networks. Chapter 3 describes the experimental setup and data acquisition, and Chapter 4 presents the analytical calibration. Chapter 5 addresses neural network design and training, while Chapter 6 covers embedded implementation and verification. Chapter 7 investigates continual learning, and Chapter 8 presents the conclusions and possible future developments.

Chapter 2

Accelerometers and Neural Network Calibration

2.1 Operating Principles of Accelerometers

An accelerometer is an inertial sensor that provides information about acceleration along one or more sensitive axes. Its operation can be described by a proof mass connected to the device structure through elastic elements, whose deformation is converted into an electrical signal by a transduction system [1]. The single-axis model in Figure 2.1(a) comprises a mass M, a spring of stiffness k and a viscous damper with coefficient c_v. To describe its behaviour, consider a translation of the structure along the sensitive axis: $y(t)$ denotes its displacement relative to an inertial reference frame, and $x(t)$ the displacement of the mass relative to the structure, measured from the reference configuration represented by l_0 in the diagram. Taking the same positive direction for both displacements, the acceleration of the structure is $a = \ddot{y}$, while the absolute acceleration of the mass is $a + \ddot{x}$. In the absence of a gravitational component along the axis, the elastic force $-kx$ and viscous force $-c_v\dot{x}$ act on the mass.

Newton's second law, $M(a+\ddot{x}) = -kx - c_v\dot{x}$, therefore gives

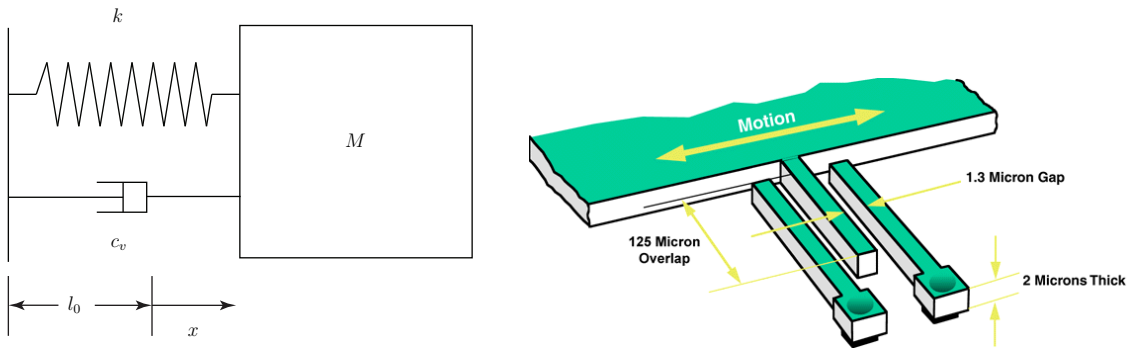
$$M\ddot{x} + c_v\dot{x} + kx = -Ma. \tag{2.1}$$

Dots denote time derivatives, while $-Ma$ represents the inertial force associated with the acceleration of the structure when motion is described relative to it. It is therefore necessary to distinguish a, the model input, from $\ddot{x}$, which describes only the relative acceleration of the mass [1].

When the imposed acceleration varies sufficiently slowly relative to the system's dynamic response and the initial transient has decayed, the contributions $M\ddot{x}$ and $c_v\dot{x}$ can be neglected relative to the elastic term, giving

$$x \simeq -\frac{M}{k}\,a. \tag{2.2}$$

This quasi-static approximation concerns the relative motion of the mass and does not require the acceleration of the structure to be zero. For example, if a constant acceleration a_0 were imposed on the structure, after the transient the mass would maintain the relative displacement $x = -Ma_0/k$ and accelerate together with the structure. Thus $\dot{x}$ and $\ddot{x}$ would be zero, while the absolute acceleration of the mass would remain a_0. In this example, spring deformation would provide the force needed to accelerate the mass, allowing acceleration to be inferred from the measured displacement. For more



(a) Mechanical model. *(b) Capacitive sensing.*

Figure 2.1: Mass–spring–damper model [1] and differential capacitive transduction [2].

rapidly varying inputs, the full dynamic model is required, since relative inertia and damping affect the sensor response.

A common implementation is the MEMS accelerometer, where MEMS stands for *Micro-Electro-Mechanical Systems*. Its mass and elastic elements are micromechanical structures generally fabricated from silicon. In capacitive devices, motion is detected through the change in capacitance between movable electrodes attached to the mass and fixed electrodes. In the differential configuration in Figure 2.1(b), displacement changes the capacitances on opposite sides of the moving element in opposite directions. The readout circuit processes this imbalance and produces a signal related to acceleration [2]. The output may be analogue or digital, depending on the electronics integrated into the sensor. Their response to constant inputs makes capacitive accelerometers particularly suitable for static measurements, as well as motion and vibration sensing [3].

Interpreting the measurements requires a distinction between the body's acceleration and the quantity detected by the sensor, known as specific force. In the ideal model, the latter is the difference between the body's acceleration and gravitational acceleration, expressed in the same reference frame. An accelerometer at rest on a surface therefore returns a non-zero measurement, whereas its ideal output in free fall is zero [4]. For an ideal triaxial sensor, let $\boldsymbol{a}_\mathrm{s} = [a_x\ a_y\ a_z]^\mathsf{T}$ be the vector of components measured along three orthogonal axes. Under static conditions,

$$\|\boldsymbol{a}_\mathrm{s}\| = \sqrt{a_x^2 + a_y^2 + a_z^2} = g_\mathrm{loc}, \tag{2.3}$$

where g_loc is the magnitude of local gravitational acceleration. If one axis is vertical, its component is $+g_\mathrm{loc}$ or $-g_\mathrm{loc}$, depending on its direction, while the other two are zero. Tilting the sensor changes the components but leaves the norm unchanged. This property allows inclination relative to the vertical to be estimated, whereas rotation about the vertical cannot be determined from an accelerometer alone. During motion, the sensor detects the accelerations acting on the body, which can therefore affect the inclination estimate [5].

In this work, accelerations are expressed in $\mathrm{m/s^2}$. The adopted conventional reference is $g_\mathrm{n} = 9.80665\ \mathrm{m/s^2}$, corresponding to the unit denoted by $1\,g$. This value is defined by convention and must be distinguished from local gravity [6]. The constancy of the norm under static conditions is the physical principle underlying the calibration developed in the following chapters. Real measurements nevertheless depart from ideal behaviour because of sensor errors and measurement noise, discussed in the next section.

2.2 Measurement Errors and Calibration

A real accelerometer departs from the ideal model because of the characteristics of its sensing structure and readout electronics. The principal systematic errors include bias and scale factor error. Bias, also called offset, is the output contribution present when the specific-force component along the axis under consideration is zero. Under static conditions, this occurs when the sensitive axis is perpendicular to the vertical. Scale factor error, by contrast, concerns the difference between the actual sensitivity and the nominal sensitivity used to convert the output into acceleration. For a single axis, neglecting other contributions, these effects can be represented as

$$a_{\mathrm{m}} = (1 + \varepsilon_s)\, a_{\mathrm{s}} + b, \tag{2.4}$$

where a_{m} is the measurement already expressed in $\mathrm{m/s^2}$, a_{s} is the ideal specific-force component, b is the bias, also in $\mathrm{m/s^2}$, and ε_s is the dimensionless relative scale factor error. Bias shifts the input–output characteristic, whereas scale factor error changes its slope. This distinction matters because correcting the zero alone does not eliminate a sensitivity error [5].

For triaxial accelerometers, the geometry of the sensitive axes and their response to transverse inputs must also be considered. The actual axes may not be perfectly orthogonal, causing errors in the reconstruction of the measured vector components. Cross-axis sensitivity instead describes the response of a channel to acceleration applied perpendicular to its nominal axis. Both effects can contribute to coupling between components, but they are not synonymous: non-orthogonality concerns axis geometry, whereas cross-axis sensitivity characterises the device response. A further distinction concerns misalignment of the entire sensor relative to the system on which it is mounted. A common rotation of the three axes changes their orientation relative to the body without changing their mutual orthogonality [7].

Many accelerometers are calibrated at the factory by the manufacturer, who determines compensation parameters that, in digital devices, can be stored and applied internally before output data become available. Subsequent conversion of the digital counts using nominal sensitivity expresses the measurement in physical units but does not re-identify errors in the individual device. Factory calibration may be adequate for an application while still leaving residual errors; moreover, stresses introduced by soldering and assembly can alter the device response. When higher accuracy is required, additional characterisation under the intended operating conditions and correction of the available measurements may therefore be necessary [8]. In this work, calibration thus refers to the developed procedure applied in addition to the factory compensation already present in the device.

Calibration must be distinguished from noise reduction: measurement noise causes fluctuations even under unchanged conditions. Averaging static acquisitions can attenuate the random component but does not automatically eliminate bias and scale factor errors. These errors may also vary with temperature and mechanical mounting conditions [9]. A calibration therefore describes sensor behaviour within the conditions and assumptions under which it was obtained and does not guarantee equal accuracy in every operating situation. Here, the reference consists of static acquisitions and the norm constraint introduced in the previous section. Parameter identification and verification of the correction are developed in the chapter on analytical calibration.

2.3 Neural Networks for Calibration

An artificial neural network is a mathematical model that represents a relationship between inputs and outputs through parameters determined during training. One of the simplest architectures is the multilayer perceptron (MLP). An MLP comprises inputs, one or more hidden layers and an output layer. In fully connected layers, each neuron combines the previous layer's outputs using coefficients called weights, adds a constant term called a bias, and applies an activation function. The network bias is a mathematical parameter and is distinct from the accelerometer's zero error described above. Information flows from inputs to outputs without feedback connections; the network is therefore termed *feedforward* [10]. Figure 2.2 shows an example with one hidden layer.

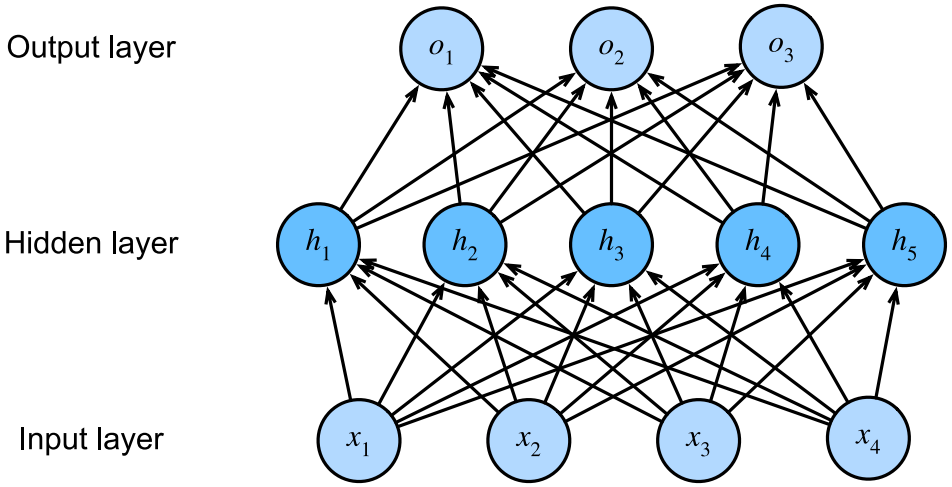


Figure 2.2: Example of an MLP with one hidden layer [10].

For a network with one hidden layer and a linear output, the computation can be written as

$$\begin{aligned} \mathbf{h} &= \varphi(\mathbf{W}_1\mathbf{u} + \mathbf{b}_1)\,, \\ \hat{\mathbf{y}} &= \mathbf{W}_2\mathbf{h} + \mathbf{b}_2, \end{aligned} \tag{2.5}$$

where $\mathbf{u}$ is the input vector, $\mathbf{h}$ the hidden activation vector and $\hat{\mathbf{y}}$ the predicted output. Matrices $\mathbf{W}_1$ and $\mathbf{W}_2$ contain the weights, while $\mathbf{b}_1$ and $\mathbf{b}_2$ are bias vectors. The function φ, applied componentwise, introduces nonlinearity; without it, the composition of the two layers would remain an affine transformation [10].

In supervised learning, each dataset input is associated with a reference output, called a target. Training adjusts weights and biases to reduce a loss function that quantifies the discrepancy between predictions and targets. For regression, where outputs are continuous numerical quantities, one possible choice is mean squared error:

$$\mathscr{L} = \frac{1}{Nq}\sum_{i=1}^{N} \left\|\hat{\mathbf{y}}_i - \mathbf{y}_i\right\|_2^2, \tag{2.6}$$

where N is the number of examples, q the number of components in each output and $\mathbf{y}_i$ the target associated with example i. The gradient of the loss with respect to the parameters is computed by *backpropagation*, applying the chain rule. An optimisation algorithm then uses these derivatives to update the parameters. The procedure is repeated over the training data; one complete pass through them is called an epoch.

Reducing training error alone does not guarantee generalisation, namely the model's ability to produce accurate results on new measurements. When a network fits the particularities of its training data too closely, its error on those data can continue to decrease while performance on other data deteriorates; this is known as *overfitting*. A validation set allows configurations to be compared and learning progress to be monitored. Since these evaluations influence model selection, any test set must also remain separate from this process [10].

In the supervised approach developed here, inputs are derived from the three measured accelerometer components, and targets are the corresponding components corrected by analytical calibration. The network thus solves a three-output regression problem, learning to approximate the reference transformation. Once the analytical calibration parameters have been identified, this transformation is affine. The use of an MLP is therefore motivated not by a need to represent an inherently nonlinear relationship, but, as explained in the introduction, by the objective of reproducing it with a compact neural model. Agreement with the targets measures fidelity to the analytical calibration and does not, by itself, independently establish the physical accuracy of the measurements. Accordingly, the comparison between the two methods is distinguished below from evaluation against the gravitational reference.

2.4 Neural Networks on Microcontrollers

As outlined above, using a neural network in an embedded system allows data to be processed directly on the device that acquires them. For an accelerometer, the microcontroller can receive the measured components and execute the model to obtain corrected outputs without requiring an external computer for this calculation. This phase, called inference, uses previously identified parameters to compute the output for a new input [10] and must be distinguished from training, which determines those parameters. In this work, the supervised network is trained on a computer and then transferred to the STM32 board, with weights and biases held fixed during inference.

Execution on a microcontroller requires a model compatible with the available resources. Memory must accommodate the code, network parameters and intermediate computational data, while processing time must suit the required output rate. In a typical implementation, code and constant parameters reside in non-volatile memory, whereas inputs, outputs and intermediate activations occupy RAM. Architecture selection must therefore consider memory use and computational cost as well as prediction quality [11]. Assessing the complete system also requires accounting for acquisition time and other program operations, rather than inference duration alone.

Transferring a model to an embedded application, known as deployment, involves converting the network into a form executable on the device and integrating it into the firmware, the program that controls device operation. In the STM32 ecosystem, tools such as STM32Cube.AI generate optimised code from trained models and analyse their memory requirements and execution times [11]. Correct integration also requires the same input preparation used on the computer, including component ordering and any normalisation operations. Comparing outputs for identical inputs therefore verifies numerical consistency between the original model and its board implementation [11].

Chapter 3

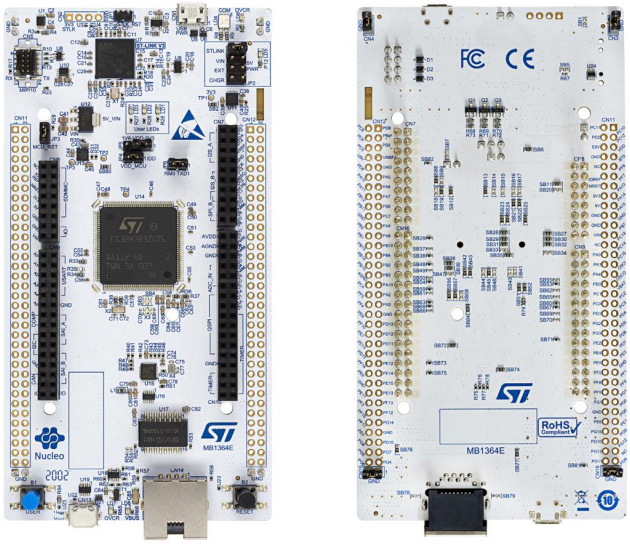
Experimental Setup and Data Acquisition

3.1 Sensors and the STM32 Nucleo Board

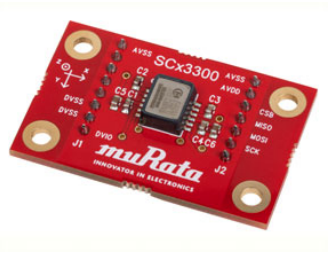
The equipment used in this thesis consists of a Murata SCL3300-D01 sensor mounted on an SCL3300-D01-PCB module and an STMicroelectronics NUCLEO-H743ZI2 board. The sensor measures the three acceleration components, while the microcontroller manages acquisition and transmission to the computer. Both components are shown in Figure 3.1. The manufacturer describes the SCL3300-D01 as a triaxial inclinometer with angle output and a digital Serial Peripheral Interface (SPI). It uses a capacitive sensing structure based on 3D-MEMS technology and an integrated circuit for signal conditioning and processing. In addition to angle outputs, acceleration components along the three axes are available through signed 16-bit digital registers; these components are used here for both analytical calibration and neural network development. The sensor offers several operating modes that change its sensitivity and internal filter bandwidth. The selected configuration is described in the acquisition firmware section. The datasheet specifies a supply range of 3.0–3.6 V and an operating temperature range of −40 to +125 °C [12]. The SCL3300-D01-PCB module exposes the terminals needed to connect external electronics [13]. The NUCLEO-H743ZI2 belongs to the Nucleo-144 family and is based on an STM32H743ZIT6 microcontroller with a 32-bit Arm Cortex-M7 processor. It provides 2 MB of Flash, approximately 1 MB of RAM and a hardware floating-point unit, useful for network computations [14]. The board includes an STLINK-V3E programmer and debugger, allowing firmware to be uploaded and execution controlled through a single USB connection. Expansion connectors expose power supplies and microcontroller signals for connection to the sensor module [15].

3.2 Connections and Software Tools

The SCL3300-D01-PCB module is connected to the NUCLEO-H743ZI2 through power lines and SPI. The board's 3.3 V output supplies AVDD and DVIO, while AVSS and DVSS connect to the common ground (GND). Communication uses the microcontroller's SPI1 peripheral configured as master. SCK provides the data-transfer clock, MOSI carries commands from the board to the sensor, and MISO returns sensor responses. The active-low CSB signal selects the device during communication and is controlled by the firmware through a digital output [12]. Table 3.1 summarises the connections and



(a) NUCLEO-H743ZI2: front and rear [16].



(b) Murata module [13].

Figure 3.1: Development board and sensor module used in the experimental setup.

their functions. The mapping between Nucleo terminals and STM32 pins is given in the board manual [15].

Table 3.1: Connections between the NUCLEO-H743ZI2 and the SCL3300-D01-PCB module.

Nucleo terminal	STM32 pin	Murata terminal	Function
3V3	–	AVDD and DVIO	Module power supply
GND	–	AVSS and DVSS	Common ground
D13	PA5	SCK	SPI communication clock
D12	PA6	MISO	Data from sensor to board
D11	PB5	MOSI	Commands from board to sensor
D10	PD14	CSB	Active-low sensor selection

Throughout all acquisition campaigns, the sensor remained in Mode 1, which uses an internal first-order low-pass filter with a nominal −3 dB cutoff frequency of 40 Hz [12]. SPI1 is configured with an idle-low clock and bit sampling on the first clock edge, corresponding to the rising edge. The STM32 peripheral transfers eight-bit units, most significant bit first; the sensor protocol uses 32-bit frames, each comprising four bytes [12].

A single USB cable connects the board's STLINK-V3E interface to the computer. The microcontroller transmits data through USART3, configured asynchronously at 115200 baud with eight data bits, no parity, one stop bit and no hardware flow control. Data reach the computer through the STLINK virtual serial port, COM5 in this work. The YAT (*Yet Another Terminal*) application was used to receive these data. USART transmit and receive signals are assigned to PD8 and PD9 respectively; their connection to STLINK is internal to the board and requires no additional external wiring [15]. During calibration acquisition, each transmitted line contains the three comma-separated digital sensor

counts. These already include internal device compensation but have not yet been converted to m/s^2.

Firmware was developed in STM32CubeIDE version 1.18.1, used for peripheral configuration, compilation, programming and board debugging. Pose recording was handled by Python scripts run from the command prompt. These access the serial port through pySerial, recognise lines containing three integers, discard lines that do not match the expected format and save valid samples to a pose-specific CSV file. Since the three campaigns share the same collection procedure, the common code is provided in Appendix A as `acquire_pose.py`; the sample count, pose name and destination folder are set according to the campaign.

3.3 Acquisition Procedure

Measurements were acquired with the sensor stationary in different orientations. The Murata module was mounted on a breadboard attached to supports designed and 3D-printed by Duchi and Ida' [17], as illustrated in their work on inertial sensor calibration. The Nucleo board rested separately and was connected by cables long enough to allow repositioning and movement of the supports between poses without pulling on the module.

The supports were moved and tilted manually to obtain different orientations, seeking to cover as many directions of the gravitational vector relative to the sensor axes as possible. Besides poses with one approximately vertical axis, intermediate orientations were included without measuring their angles. This is consistent with the multiposition calibration principle described by Tedaldi, Pretto and Menegatti [18]: static measurements can be used without knowing each pose's orientation, provided that the poses are sufficiently numerous and distributed across different spatial orientations.

Once the sensor had been positioned, the acquisition script was started. After an initial five-second wait to allow stabilisation, the serial buffer was cleared to exclude data accumulated during that interval. Valid samples were then collected while keeping the sensor stationary until the required count was reached. The samples were saved to a dedicated pose file, retaining the digital counts for all three components. The sensor was then rotated into a different orientation and the procedure repeated.

3.4 Data Organisation

The acquisitions were organised into three groups of static poses according to their purpose: identifying the analytical calibration, expanding the neural network dataset and performing the final comparison between methods. Individual samples, consisting of the three digital counts returned by the sensor, were retained for each pose, with data from different acquisitions kept separate.

The first campaign comprises 26 poses, C01–C26, used to identify the analytical calibration parameters. Each contains 1500 samples, giving 39 000 triplets in total. Table 3.2 lists the intended orientations: signs indicate the axis directions pointing upwards without specifying the actual angle; in intermediate poses, the different axes need not

have equal inclinations to the vertical. The second campaign comprises 50 additional static poses acquired at random orientations, each containing 500 samples, for a total of 25 000 triplets. These acquisitions were combined with the 26 C poses, from which a subset of 500 samples per pose is subsequently extracted as described in Chapter 5, to build the neural network development dataset. Overall, 76 poses and 64 000 samples were acquired. Each input triplet was associated with the corresponding analytical calibration output as the supervised learning reference. The dataset was split into *training* and *validation* sets, keeping all samples from a given pose in the same subset. Data preparation and splitting criteria are discussed in the dedicated chapter. Finally, the third campaign comprises 12 poses, V01–V12, with 1500 samples per pose and 18 000 triplets overall. During analytical calibration and supervised network development, these acquisitions remained separate from parameter identification, training and model selection. They were used only in the final evaluation to compare nominal factory conversion, analytical calibration and neural network measurements. For these two stages, the V poses are therefore termed the external test set, distinct from the *validation* set used during training. Their subsequent use in continual learning development is discussed in the relevant chapter. Table 3.3 summarises the size and purpose of the three campaigns.

Table 3.2: Orientations of the poses used for analytical calibration.

Pose	Axis directions pointing upwards
C01	$+X$
C02	$-X$
C03	$+Y$
C04	$-Y$
C05	$+Z$
C06	$-Z$
C07	$+X, +Y$
C08	$+X, -Y$
C09	$-X, +Y$
C10	$-X, -Y$
C11	$+X, +Z$
C12	$+X, -Z$
C13	$-X, +Z$
C14	$-X, -Z$
C15	$+Y, +Z$
C16	$+Y, -Z$
C17	$-Y, +Z$
C18	$-Y, -Z$
C19	$-X, +Y, +Z$

Table 3.2: continued.

Pose	Axis directions pointing upwards
C20	$-X, -Y, -Z$
C21	$-X, -Y, +Z$
C22	$+X, +Y, -Z$
C23	$-X, +Y, -Z$
C24	$+X, +Y, +Z$
C25	$+X, -Y, +Z$
C26	$+X, -Y, -Z$

Table 3.3: Size and purpose of the acquisition campaigns.

Campaign	Pose	Samples per pose	Samples in total	Purpose
Poses C01–C26	26	1500	39 000	Analytical calibration and neural network development
Additional static poses	50	500	25 000	Neural network development
Poses V01–V12	12	1500	18 000	External test and final comparison

Chapter 4

Analytical Calibration of the Accelerometer

4.1 Mathematical Model

The analytical calibration follows the multiposition model proposed by Tedaldi, Pretto and Menegatti [18], using only the accelerometer formulation presented in Sections II and III of their work. This framework estimates bias, scale factor and non-orthogonality contributions from the static measurements described in the previous chapter, exploiting the constraint on the specific-force norm. Here, the transformation is applied to the digital counts supplied by the Murata SCL3300-D01 and therefore provides an additional correction beyond the sensor's internal compensation.

Let $\mathbf{r} = [r_x \; r_y \; r_z]^\mathsf{T}$ denote the digital count vector. The calibrated measurement is

$$\mathbf{a}_{\text{cal}} = \mathbf{T}_a \mathbf{K}_a (\mathbf{r} + \mathbf{b}), \tag{4.1}$$

where $\mathbf{a}_{\text{cal}}$ contains the three corrected components in m/s^2. Vector $\mathbf{b}$ corrects offsets, diagonal matrix $\mathbf{K}_a$ applies the conversion coefficients for the three axes, and matrix $\mathbf{T}_a$ corrects their non-orthogonality. The operation order matters: offsets are first compensated in counts, scale factors are then applied, and finally the non-orthogonality matrix combines the components. These quantities are defined as

$$\mathbf{b} = \begin{bmatrix} b_x \\ b_y \\ b_z \end{bmatrix}, \qquad \mathbf{K}_a = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & s_z \end{bmatrix}, \qquad \mathbf{T}_a = \begin{bmatrix} 1 & -\alpha_{yz} & \alpha_{zy} \\ 0 & 1 & -\alpha_{zx} \\ 0 & 0 & 1 \end{bmatrix}. \tag{4.2}$$

The components of $\mathbf{b}$ are expressed in digital counts because they are added directly to the readings. The positive sign follows the adopted convention: $\mathbf{b}$ is the corrective term and therefore has the opposite sign to the offset being compensated. It is thus distinct from the acceleration bias introduced in the measurement model in Section 2.2. Coefficients s_x, s_y and s_z are expressed in m/s^2 per count and incorporate both conversion to physical units and correction of individual-axis sensitivity; they are not dimensionless factors. Parameters α_{yz}, α_{zy} and α_{zx} are dimensionless and describe the model's coupling terms for small departures from orthogonality. Matrix $\mathbf{T}_a$ is upper triangular, following the reference parametrisation, and is not a rotation matrix. Its structure establishes a convention for expressing the corrected measurement without determining the absolute orientation of the entire sensor relative to an external reference.

There are nine unknowns, collected in the vector

$$\boldsymbol{\theta} = \begin{bmatrix} \alpha_{yz} & \alpha_{zy} & \alpha_{zx} & s_x & s_y & s_z & b_x & b_y & b_z \end{bmatrix}^\mathsf{T}. \tag{4.3}$$

The product $\mathbf{C} = \mathbf{T}_a\mathbf{K}_a$ is the overall calibration matrix. Once the parameters are determined, the transformation can be rewritten as

$$\mathbf{a}_{\text{cal}} = \mathbf{C}\mathbf{r} + \mathbf{C}\mathbf{b}. \tag{4.4}$$

As noted above, this is an affine transformation of the digital counts with constant coefficients. The model contains no temperature or time dependence: the parameters represent the sensor's behaviour under the acquisition conditions used for estimation. Noise remains in the measurements and is not removed by the transformation; its effect on estimation is reduced by averaging each pose, as explained below.

Parameter identification exploits the static constraint introduced in Section 2.1. For an ideal stationary accelerometer, the specific-force norm equals the magnitude of local gravity. The processing here adopts the conventional reference $g_n = 9.80665\,\text{m/s}^2$ without experimentally determining local gravity. The constraint imposed on the corrected measurement is therefore

$$\|\mathbf{a}_{\text{cal}}\|_2 \simeq g_n. \tag{4.5}$$

Orientation changes the individual components but not the norm expected by the model. The estimation problem can thus be formulated without assigning a known acceleration vector to each pose. Let $\bar{\mathbf{r}}_i$ be the mean count vector for pose i; its calibrated measurement is

$$\bar{\mathbf{a}}_{\text{cal},i}(\boldsymbol{\theta}) = \mathbf{T}_a(\boldsymbol{\theta})\mathbf{K}_a(\boldsymbol{\theta})\big(\bar{\mathbf{r}}_i + \mathbf{b}(\boldsymbol{\theta})\big). \tag{4.6}$$

To retain accelerations in m/s^2 while preserving the normalisation used in the script, the residual is defined as

$$e_i(\boldsymbol{\theta}) = \frac{g_n^2 - \big\|\bar{\mathbf{a}}_{\text{cal},i}(\boldsymbol{\theta})\big\|_2^2}{g_n^2}. \tag{4.7}$$

The residual is dimensionless and vanishes when the squared norm equals the reference. This expression is equivalent to the implementation, in which accelerations are normalised by the gravitational reference before evaluating the residual. It is not simply the difference between two norms: the discrepancy is calculated between their squares.

The parameters are estimated by minimising the sum of squared residuals over the $N_C = 26$ calibration poses:

$$\begin{aligned} \boldsymbol{\theta}^* &= \arg\min_{\boldsymbol{\theta}} \frac{1}{2}\sum_{i=1}^{N_C} e_i(\boldsymbol{\theta})^2, \\ e_i(\boldsymbol{\theta}) &= 1 - \frac{\big\|\mathbf{T}_a(\boldsymbol{\theta})\mathbf{K}_a(\boldsymbol{\theta})\big(\bar{\mathbf{r}}_i + \mathbf{b}(\boldsymbol{\theta})\big)\big\|_2^2}{g_n^2}. \end{aligned} \tag{4.8}$$

Each pose contributes one equally weighted residual. The problem is nonlinear in the unknown parameters, which appear in matrix products and the squared norm, although the transformation applied to measurements becomes affine once the coefficients are fixed. The solution is sought with the iterative Levenberg–Marquardt algorithm. The limitations of the norm constraint are discussed in Section 4.4.

4.2 Parameter Identification

The nine parameters were identified using only poses C01–C26, described in the previous chapter. The calibration code is provided in Appendix B. A mean digital count vector

was calculated for each acquisition to attenuate fluctuations in individual samples. Denoting sample j of pose i by $\mathbf{r}_{i,j}$, the mean is

$$\bar{\mathbf{r}}_i = \frac{1}{M}\sum_{j=1}^{M}\mathbf{r}_{i,j}, \qquad M = 1500. \tag{4.9}$$

The optimisation set therefore consists of 26 mean vectors, each generating a residual according to the preceding model. Averaging reduces the variable noise component but does not eliminate systematic errors, which must be identified and corrected by calibration. Since the transformation is affine, applying it to the mean vector is equivalent to averaging the transformed vectors for the same parameters. This equivalence does not extend to the norm: the adopted procedure uses the norm of the mean vector, not the mean of individual sample norms.

The algorithm requires an initial parameter estimate, constructed from the six axial poses C01–C06. For each axis $k \in \{x, y, z\}$, let $\overline{r}_{k,+}$ and $\overline{r}_{k,-}$ be the mean components recorded in the two poses with opposite axis directions pointing upwards. Initially assuming ideal vertical alignment, the expected components are $+g_n$ and $-g_n$. The initial scale coefficient and corrective term are therefore

$$s_k^{(0)} = \frac{2g_n}{\overline{r}_{k,+} - \overline{r}_{k,-}}, \qquad b_k^{(0)} = -\frac{\overline{r}_{k,+} + \overline{r}_{k,-}}{2}. \tag{4.10}$$

The first ratio relates the count range to the theoretical acceleration difference between the poses; the second shifts the midpoint of the opposite readings to the origin. Consistently with Section 4.1, $s_k^{(0)}$ is in $\mathrm{m/s^2}$ per count and $b_k^{(0)}$ in digital counts. Since orientations were set manually, ideal alignment is only an approximation used to initialise optimisation. The axial poses are not treated as exact angular references during subsequent minimisation.

The three non-orthogonality parameters are initialised to zero. The initial vector is therefore

$$\boldsymbol{\theta}^{(0)} = \begin{bmatrix} 0 & 0 & 0 & s_x^{(0)} & s_y^{(0)} & s_z^{(0)} & b_x^{(0)} & b_y^{(0)} & b_z^{(0)} \end{bmatrix}^{\mathsf{T}}. \tag{4.11}$$

Starting from this estimate, all nine parameters are jointly updated using the 26 poses, while the V poses remain excluded throughout calibration. In the implementation, scale coefficients operate on accelerations normalised by g_n; the expressions presented here incorporate the same reference into the coefficients to return accelerations directly in $\mathrm{m/s^2}$. This changes the parameter representation without performing a new identification.

Minimisation was performed in Python with SciPy's `least_squares` function using the Levenberg–Marquardt method, which determines updates from the residual vector and its variation with the parameters. This dependence is described by the Jacobian matrix

$$\mathbf{J}(\boldsymbol{\theta}) = \left[\frac{\partial e_i}{\partial \theta_j}\right]_{\substack{i=1,\dots,26 \\ j=1,\dots,9}} \in \mathbb{R}^{26\times 9}. \tag{4.12}$$

The Jacobian is approximated numerically using two-point finite differences. Variable scaling based on the norms of its columns is also enabled, which is useful because the parameters have different orders of magnitude. The objective remains the sum

Table 4.1: Calibration parameter optimisation settings.

Setting	Selected value
Algorithm	Levenberg–Marquardt
Jacobian calculation	Two-point finite differences
Variable scaling	Adapted using the Jacobian
Objective function	Ordinary least squares
Tolerances `ftol`, `xtol`, `gtol`	10^{-12} each
Function evaluation limit (`max_nfev`)	10 000

of squared residuals defined in Section 4.1. Table 4.1 summarises the settings; their meaning is explained in the official SciPy documentation [19].

The tolerances govern numerical stopping criteria and do not guarantee a corresponding accuracy of the corrected accelerations. Nor does satisfying a convergence criterion prove that the global minimum has been found. The script therefore performs additional checks after optimisation, verifying that the parameters are finite, all three scale factors are positive and the final Jacobian has full rank, nine in this case. Positivity of the scale factors is checked after estimation rather than imposed as a minimisation constraint. Rank is assessed from the Jacobian's singular values. Writing these as $\sigma_1 \geq \cdots \geq \sigma_9$, the script treats values above the threshold

$$\tau = \max(26, 9)\, \varepsilon_{\text{mach}}\, \sigma_1, \tag{4.13}$$

as non-zero, where $\varepsilon_{\text{mach}}$ is machine precision for the numerical representation used. The numerical analysis also computes the condition number

$$\kappa_2(\mathbf{J}) = \frac{\sigma_1}{\sigma_9}. \tag{4.14}$$

Full rank indicates that, in the adopted parametrisation and at the numerical precision of the calculation, the nine Jacobian columns are linearly independent. The condition number indicates local problem sensitivity but also depends on parameter scales and units. As these checks concern only numerical aspects of the solution, experimental verification remains necessary; its results follow.

4.3 Validation and Results

Identification yielded the nine parameters in Table 4.2. Scale coefficients are in $\mathrm{m/s^2}$ per count, consistently with Section 4.1; offset correction terms remain in digital counts, and non-orthogonality parameters are dimensionless. The tabulated values are rounded for clarity, whereas processing used coefficients saved at the available numerical precision.

Verification used twelve poses, V01–V12, that had not been used to identify the parameters. For each pose, the mean count vector was calculated and separately processed by nominal conversion, denoted Factory, and the analytical transformation, denoted Calibrated. In Mode 1, the Factory conversion used is

$$\bar{\mathbf{a}}_{\mathrm{F},i} = c_{\mathrm{F}}\, \bar{\mathbf{r}}_i, \qquad c_{\mathrm{F}} = \frac{9.80665}{6000}\, \frac{\mathrm{m/s^2}}{\mathrm{count}}. \tag{4.15}$$

Table 4.2: Identified calibration parameters for the Murata SCL3300-D01.

Parameter	Value	Unit
α_{yz}	$-2.32846275748 \times 10^{-3}$	Dimensionless
α_{zy}	$-2.50986271110 \times 10^{-3}$	Dimensionless
α_{zx}	$-1.75689392955 \times 10^{-3}$	Dimensionless
s_x	$1.63811179207 \times 10^{-3}$	m/s^2 per count
s_y	$1.63945724695 \times 10^{-3}$	m/s^2 per count
s_z	$1.63852455056 \times 10^{-3}$	m/s^2 per count
b_x	0.275886880098	Counts
b_y	21.4584963405	Counts
b_z	43.4102055560	Counts

The denominator follows from the sensor's nominal sensitivity [12], while the numerator is the adopted conventional reference for expressing acceleration in physical units. Calibrated measurements instead use the parameters in Table 4.2:

$$\bar{\mathbf{a}}_{\mathrm{C},i} = \mathbf{T}_a \mathbf{K}_a \left(\bar{\mathbf{r}}_i + \mathbf{b} \right). \tag{4.16}$$

Both methods therefore use identical data and the same gravitational reference. The comparison concerns norms of pose mean vectors, not the dispersion of individual samples within each acquisition.

For each method, the norm deviation in m/s^2 is defined as

$$d_i = \left\| \bar{\mathbf{a}}_i \right\|_2 - g_n, \tag{4.17}$$

where $\bar{\mathbf{a}}_i$ is the mean vector obtained with Factory or Calibrated. Positive deviations indicate a norm above the reference, and negative deviations a norm below it. Over the $N_V = 12$ poses, mean absolute error, root mean squared error and maximum absolute error were calculated, all in acceleration units:

$$\mathrm{MAE} = \frac{1}{N_V} \sum_{i=1}^{N_V} |d_i|, \tag{4.18}$$

$$\mathrm{RMSE} = \sqrt{\frac{1}{N_V} \sum_{i=1}^{N_V} d_i^2}, \qquad e_{\max} = \max_{i=1,\ldots,N_V} |d_i|. \tag{4.19}$$

MAE describes the average absolute deviation, RMSE gives greater weight to larger deviations, and maximum absolute error identifies the worst case among the poses considered. For each metric, percentage reduction is the decrease divided by the corresponding Factory value.

The results show a reduction exceeding 96% for all metrics. RMSE decreases from 0.059047 to 0.002028 m/s^2, while maximum absolute deviation falls from 0.098703 to 0.003330 m/s^2. Improvement on new poses never used to determine the nine parameters indicates that the analytical calibration does more than fit the estimation acquisitions: it is also effective for the additional orientations examined.

The pose-by-pose comparison in Figure 4.1 shows the distribution of deviations. Calibration reduces absolute error in eleven of the twelve poses. The exception is V04, where

Table 4.3: Norm errors over the twelve external test poses.

Metric	Factory [m/s^2]	Calibrated [m/s^2]	Reduction [%]
RMSE	0.059047	0.002028	96.57
MAE	0.047379	0.001763	96.28
$e_{\max}$	0.098703	0.003330	96.63

Factory has a deviation of approximately $+0.000181\,\mathrm{m/s^2}$ and Calibrated approximately $-0.002390\,\mathrm{m/s^2}$. Overall improvement therefore does not imply uniform improvement in every measurement. After correction, the largest deviation occurs in V11 and is approximately $0.003330\,\mathrm{m/s^2}$ in absolute value.

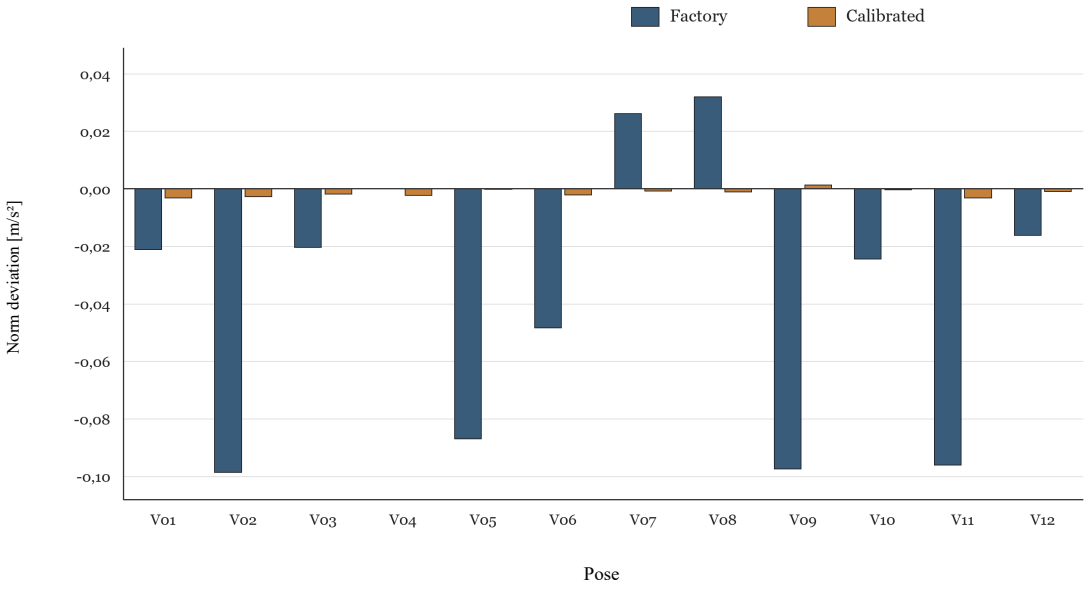


Figure 4.1: Norm deviation from the reference for poses V01–V12.

These results concern the static conditions examined and cannot be extended to dynamic conditions or different temperatures.

4.4 Directional Verification and Limitations

To examine how a norm-based correction also affects vector direction, the vectors from the six axial poses C01–C06 were analysed. A nominal direction is available for these poses, corresponding to the axis intended to be vertical. The comparison examines how and by how much calibration changes the measured direction relative to that reference. However, these poses belong to the parameter-identification set: this is a check on calibration acquisitions, not an external test such as that performed on the V poses.

For each pose, Factory and Calibrated mean vectors were calculated using the transformations in Section 4.3. Each vector's direction was isolated by normalisation:

$$\mathbf{u}_i = \frac{\bar{\mathbf{a}}_i}{\|\bar{\mathbf{a}}_i\|_2}. \tag{4.20}$$

The unit vector $\mathbf{u}_i$ is dimensionless; the components of $\bar{\mathbf{a}}_i$ are in $\mathrm{m/s^2}$. Let $\mathbf{u}_{\mathrm{rif},i}$ be the unit vector in the nominal direction. Angular deviation is defined as

$$\delta_i = \arccos\left(\mathbf{u}_i^{\mathsf{T}} \mathbf{u}_{\mathrm{rif},i}\right). \tag{4.21}$$

For C01–C06 respectively, the references are the unit vectors along $+X$, $-X$, $+Y$, $-Y$, $+Z$ and $-Z$. The dot product gives the cosine of the angle between measured and nominal directions. Numerically, its value is clipped to $[-1,1]$ before applying the inverse cosine to avoid rounding errors. The resulting angles are converted from radians to degrees for presentation.

Table 4.4 reports the deviations for both methods and their change, $\Delta\delta_i = \delta_{\mathrm{C},i} - \delta_{\mathrm{F},i}$. Negative values indicate movement towards the nominal direction after calibration; positive values indicate movement away from it. Correction reduces the deviation in C02, C03 and C04 but increases it in C01, C05 and C06. Mean angular deviation changes from 0.720536° to 0.681998°; the root mean square deviation changes from 0.748672° to 0.739844°. The maximum decreases from 1.062799° to 1.043536°. Overall changes are small and do not show a uniform directional improvement.

Table 4.4: Angular deviations from nominal directions for the axial poses.

Pose	Nominal direction	Factory [°]	Calibrated [°]	$\Delta\delta_i$ [°]
C01	$+X$	0.558153	0.794860	+0.236707
C02	$-X$	1.062799	0.613193	−0.449606
C03	$+Y$	0.476080	0.359186	−0.116894
C04	$-Y$	0.577309	0.295884	−0.281425
C05	$+Z$	0.788019	0.985330	+0.197310
C06	$-Z$	0.860856	1.043536	+0.182680

Interpreting these values requires considering sensor positioning. Reference directions describe the intended manual orientation, but were not established through independent angular measurement. The calculated deviation therefore combines sensor-response effects with actual acquisition alignment, which this test cannot separate. The reported digits describe numerical processing results, not equivalent experimental angular accuracy. In particular, a lower value after calibration indicates closer agreement with the nominal reference but does not necessarily demonstrate greater accuracy relative to the actual direction. This distinction is consistent with the identification constraint. For a rotation matrix $\mathbf{R}$,

$$\|\mathbf{Ra}\|_2 = \|\mathbf{a}\|_2. \tag{4.22}$$

The objective therefore cannot detect a common rotation of the corrected vectors relative to an external reference. The triangular calibration matrix fixes the model convention

but does not replace sensor-alignment measurement [18]. Quantitative verification of directional accuracy would require independently determined orientations with known uncertainty. Similarly, the norm errors in the preceding section are referenced to g_n; any difference from local gravity was not determined separately.

The identified transformation provides the reference for neural network training in the next chapter. Comparison with the targets will therefore assess the network's ability to reproduce this calibration, subject to the limitations discussed here.

Chapter 5

Neural Network Design and Training

5.1 Dataset Preparation

Dataset preparation pairs each sensor measurement with the analytical calibration result, which provides the supervised learning reference. Unlike parameter identification in the previous chapter, which used pose mean vectors, network training uses individual samples. Each example therefore comprises the three raw measurements $\mathbf{r}_i$ and the corresponding corrected component vector $\mathbf{a}_{\mathrm{cal},i}$ obtained from the analytical transformation:

$$\mathbf{a}_{\mathrm{cal},i} = \mathbf{T}_a \mathbf{K}_a \left(\mathbf{r}_i + \mathbf{b}\right). \tag{5.1}$$

Vector $\mathbf{r}_i$ contains the three digital counts returned by the sensor for sample i before additional calibration. The term "raw" thus refers to this processing stage, since the counts already include internal device compensation. Indeed, the datasheet states that factory calibration parameters are stored in non-volatile memory and loaded automatically at startup; the documented SPI interface provides no modes or registers for accessing measurements before this compensation [12, Sections 3.1 and 6]. Vector $\mathbf{a}_{\mathrm{cal},i}$ contains the calibrated components in $\mathrm{m/s^2}$, following Chapter 4. Calibration parameters remain fixed during both dataset construction and network training. Targets therefore represent analytical model outputs, stored in dimensionless form as

$$\mathbf{y}_i = \frac{\mathbf{a}_{\mathrm{cal},i}}{g_n}, \qquad g_n = 9.80665\,\mathrm{m/s^2}. \tag{5.2}$$

No further target standardisation is applied. Acceleration is recovered by multiplying the network output by g_n; accelerations and errors in the same units are consequently reported below in $\mathrm{m/s^2}$.

The acquisitions in Section 3.4 comprise 76 poses and 64 000 samples, but each C pose contains 1500 samples whereas each of the fifty additional poses contains 500. Using all measurements with equal weight would therefore give each C pose three times the contribution of an additional pose to the loss. To give every pose the same sample count, the development dataset retains only the first 500 samples of each C pose in acquisition order and all samples from the other poses, giving 38 000 examples.

The pose-based split assigns 64 poses to training and 12 to validation, comprising 32 000 and 6 000 samples respectively; measurements from the same acquisition therefore never appear in both sets. The test set contains the twelve V poses, with 1500 samples each and 18 000 samples overall, since these are used only for testing and do not contribute to weight updates. Table 5.1 gives the composition and role of each set. Following the methodological distinction in *Dive into Deep Learning* [10], training updates weights

and biases, validation compares architectures and selects the checkpoint, and testing takes place only after model selection.

Table 5.1: Data split for neural network development and testing.

Set	Poses	Samples	Purpose
Training	22 C and 42 additional	32 000	Weight and bias updates
Validation	4 C and 8 additional	6 000	Model comparison and selection
External test	12 V	18 000	Final evaluation with a fixed model

Before network input, digital counts are standardised separately for each axis, consistently with scikit-learn's guidance on MLP sensitivity to input scale [22]. For component $k \in \{x, y, z\}$, the standardised input is

$$x_{i,k} = \frac{r_{i,k} - \mu_k}{\sigma_k}, \tag{5.3}$$

where μ_k and σ_k are the mean and standard deviation of counts calculated on the training set alone, using the training sample count as the variance denominator. This yields dimensionless inputs with zero mean and unit variance on the training set and brings the three components to comparable scales. The same coefficients are applied to validation and test data without recalculation, and saved with the weights because every subsequent execution must reproduce the same input preparation. The formula matches *StandardScaler*, although the script implements it directly in NumPy [23].

5.2 Architectures and Training

The code for training and comparing architectures appears in Appendix C. Networks were implemented in Python using TensorFlow and its Keras interface with fully connected layers. Every configuration has three inputs, the standardised counts, and three outputs, the normalised calibrated components defined above. Tests considered one or two hidden layers and varied neuron counts to compare different complexity levels. Table 5.2 lists the architectures; each sequence gives the input count, hidden-layer neuron counts and output count in order.

Table 5.2: Architectures examined and trainable parameter counts.

Architecture	Hidden layers	Parameters
$3 \to 5 \to 3$	1	38
$3 \to 10 \to 3$	1	73
$3 \to 10 \to 5 \to 3$	2	113
$3 \to 3 \to 3 \to 3$	2	36
$3 \to 5 \to 5 \to 3$	2	68
$3 \to 8 \to 5 \to 3$	2	95

Parameter counts include connection weights and biases. A fully connected layer with n_{in} inputs and n_{out} neurons contains $n_{\text{in}} n_{\text{out}}$ weights and n_{out} biases, totalling $n_{\text{in}} n_{\text{out}} + n_{\text{out}}$ parameters, consistently with the model in Chapter 2. Hidden neurons use the hyperbolic tangent activation, whose output lies between -1 and 1:

$$\phi(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}. \tag{5.4}$$

Output neurons instead use a linear activation to avoid imposing the same bound on predicted components. For network output $\widehat{\mathbf{y}}_i$, the corresponding acceleration is $\widehat{\mathbf{a}}_{\text{cal},i} = g_n \widehat{\mathbf{y}}_i$. All architectures therefore share the same input and output representation and differ only in hidden-layer structure.

Training minimises mean squared error between predictions and targets. For N samples, the loss is

$$\mathscr{L}_{\text{MSE}} = \frac{1}{3N} \sum_{i=1}^{N} \left\| \widehat{\mathbf{y}}_i - \mathbf{y}_i \right\|_2^2. \tag{5.5}$$

The factor $3N$ averages over all samples and all three components. Since targets are normalised by g_n, the implemented loss is dimensionless. Parameters are updated with Adam, short for *Adaptive Moment Estimation*, which adapts updates using estimates of the first and second gradient moments [20]. The base learning rate was fixed at 10^{-3}, the Keras Adam default [24], and held constant throughout training, while Adam adapts individual parameter updates through gradient statistics. Training uses mini-batches of 256 samples and reshuffles the training set every epoch. Shuffling affects only the order of samples already assigned to training, preserving the pose-based split. Seed 42 was reset before constructing each model in the comparison. With 32 000 samples and batches of 256, each epoch contains 125 updates.

All architectures were allowed a maximum of 500 epochs, with early stopping based on validation MSE: training stops after 30 consecutive epochs without improvement. The weights corresponding to the observed minimum are restored, following Keras *EarlyStopping* [21]. The model at that minimum is also saved during training for subsequent evaluation. Consequently, the actual epoch count may differ between architectures, while the loss, optimiser and training settings remain common. MAE is recorded as an additional metric but affects neither stopping nor checkpoint selection. Table 5.3 summarises the settings. Results and an extended run without early stopping are presented next.

5.3 Model Selection

Architectures were compared using validation performance, evaluated for each network at its minimum-MSE checkpoint. The comparison considers MAE and RMSE pooled across all three components, together with trainable parameter count. Errors are expressed in m/s^2 by multiplying normalised output–target differences by g_n. Table 5.4 reports results under the protocol in Section 5.2.

The $3 \to 10 \to 3$ network was selected because it achieves the lowest validation MSE among the six architectures, with an RMSE of $4.100 \times 10^{-3}\,\text{m/s}^2$ and 73 parameters. The comparison uses a single run per architecture under common settings; it therefore does

Table 5.3: Common training settings.

Setting	Value or criterion
Inputs	Z-score using training statistics only
Target	Accelerations divided by g_n
Activations	Tanh hidden layers; linear output
Loss function	MSE over three normalised components
Optimiser	Adam
Base learning rate	10^{-3}, constant
Mini-batch	256 samples
Shuffling	Every epoch, training only
Seed	42
Maximum epochs	500
Early stopping	30 epochs without validation MSE improvement
Retained checkpoint	Minimum validation MSE
Additional metric	MAE

Table 5.4: Architecture comparison on the validation set.

Architecture	Parameters	MAE [m/s^2]	RMSE [m/s^2]
$3 \to 5 \to 3$	38	1.630×10^{-2}	1.938×10^{-2}
$3 \to 10 \to 3$	73	3.226×10^{-3}	4.100×10^{-3}
$3 \to 10 \to 5 \to 3$	113	4.266×10^{-3}	6.189×10^{-3}
$3 \to 3 \to 3 \to 3$	36	2.317×10^{-2}	2.943×10^{-2}
$3 \to 5 \to 5 \to 3$	68	6.695×10^{-3}	8.245×10^{-3}
$3 \to 8 \to 5 \to 3$	95	5.273×10^{-3}	8.466×10^{-3}

not establish general superiority of the selected structure. The minimum occurs at epoch 269, and training stops at epoch 299 after thirty consecutive epochs without improvement, consistently with the stopping rule. Figure 5.1 shows training and validation MSE in $(\mathrm{m/s^2})^2$, obtained by multiplying the recorded values by g_n^2. This conversion changes neither the locations of the minima nor the selection criterion.

Early stopping means that the previous minimum has not improved for thirty consecutive epochs; it does not require validation loss to increase continuously [21]. This behaviour alone therefore does not demonstrate overfitting, which would instead be suggested by decreasing training error accompanied by persistent deterioration on validation data [10]. To examine behaviour beyond the stopping point, the $3 \to 10 \to 3$ network was trained again for 2 000 epochs without early stopping, retaining the dataset, standardisation, seed, mini-batch size and 10^{-3} base learning rate. This additional run is purely diagnostic and does not replace the previously selected checkpoint.

Figure 5.2 shows the full history and a detailed view from epoch 100 onwards. The global validation minimum occurs at epoch 1926 and is lower than in the early-stopped run of the same architecture. However, oscillations require optimisation behaviour to be distinguished from deteriorating generalisation. No scheduler is present and the

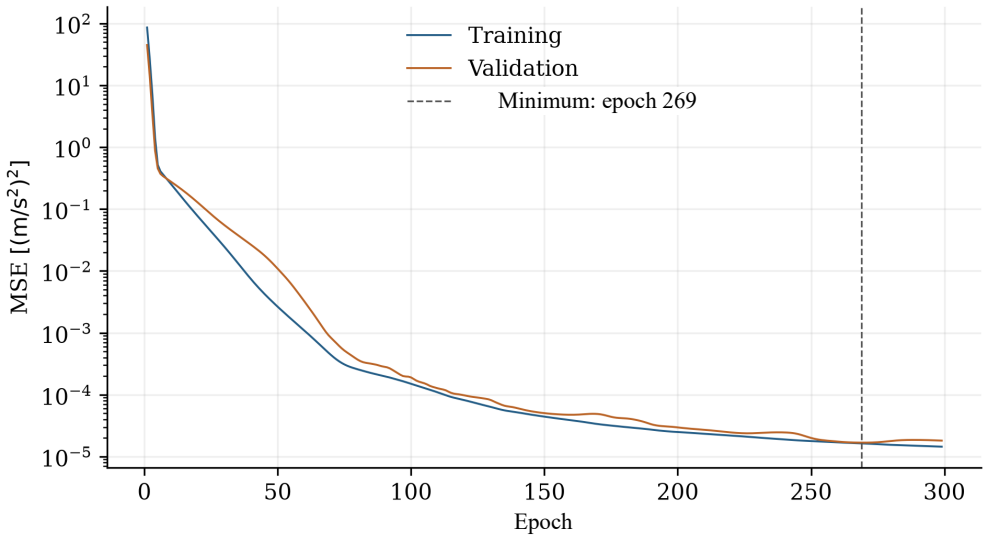


Figure 5.1: MSE of the early-stopped 3–10–3 network; validation minimum at epoch 269.

recorded learning rate remains constant, although Adam continues to alter effective parameter updates through gradient statistics [24]. Furthermore, the training MSE normally returned by Keras averages mini-batch losses evaluated while weights are being updated, whereas validation is evaluated at epoch end [25]. For a more direct comparison, the extended-run plot recalculates training error over the entire training set at the end of each epoch without updating weights, so that both curves refer to the same model.

Oscillations also occur in the training error recalculated at epoch end, while validation error does not deteriorate persistently relative to training error. Thus, over the examined interval, the curves do not progressively separate in a manner indicating excessive adaptation to training data. To investigate the update step, a further test resumed in both cases from the weights and Adam state at epoch 2000 and ran for 200 epochs with identical shuffling. One run retained a learning rate of 10^{-3}, while the other used 10^{-4}. Figure 5.3 shows the two validation curves. Over the final hundred epochs, the standard deviation of MSE decreases from 2.839×10^{-6} to 1.624×10^{-8} $(\mathrm{m/s^2})^2$. This test shows that reducing the learning rate attenuates oscillations, highlighting the role of the weight-update step. The behaviour is consistent with updates near a loss minimum that are large enough to overshoot it repeatedly, while smaller steps reduce these excursions. Zhang et al. [10] illustrate this mechanism for gradient descent; here, Adam additionally adapts updates through gradient statistics [24]. The test supports this interpretation without directly reconstructing the loss geometry. The phenomenon occurs at a constant learning rate within each run and therefore does not require a scheduler; oscillations alone do not demonstrate poorer generalisation.

5.4 Offline Evaluation

Final evaluation used the selected $3 \to 10 \to 3$ network with the weights from epoch 269. The test contains 18 000 samples across V01–V12, kept separate from both training and model selection. Input standardisation also uses the training-set parameters without recalculating them on the new data. For each sample, the network output is compared

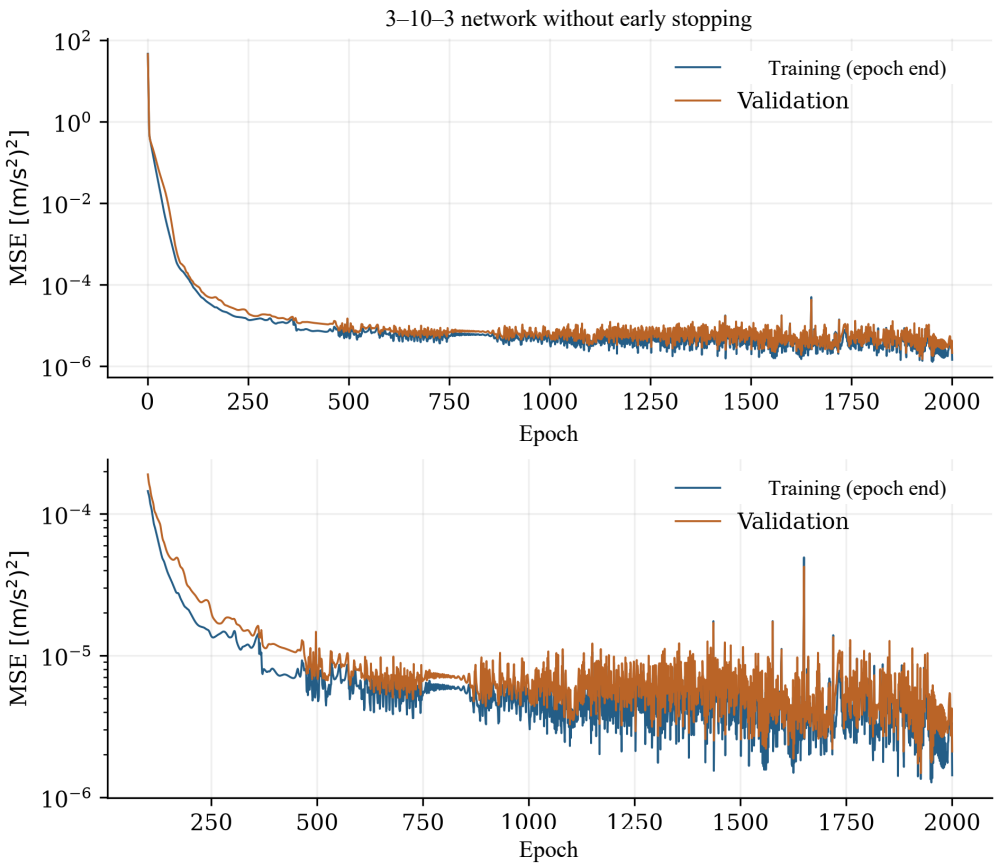


Figure 5.2: End-of-epoch MSE of the 3–10–3 network without early stopping.

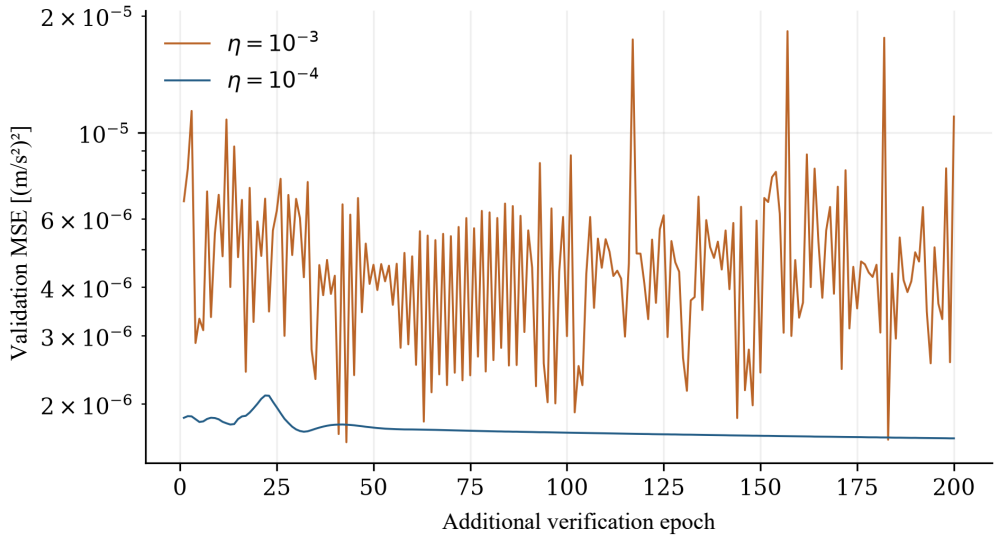


Figure 5.3: Learning-rate comparison from identical weights and Adam state.

with the analytical calibration applied to the same sensor measurement; both outputs are converted to m/s^2 before calculating errors. Let $\widehat{\boldsymbol{a}}_i$ be the network prediction and $\boldsymbol{a}_i$ the analytical reference, with $\boldsymbol{e}_i = \widehat{\boldsymbol{a}}_i - \boldsymbol{a}_i$. Overall MAE and RMSE pool the three component errors over all N samples:

$$\mathrm{MAE} = \frac{1}{3N}\sum_{i=1}^{N}\sum_{k=1}^{3}|e_{ik}|, \qquad \mathrm{RMSE} = \sqrt{\frac{1}{3N}\sum_{i=1}^{N}\sum_{k=1}^{3}e_{ik}^2}. \tag{5.6}$$

Table 5.5 reports results on individual samples without first averaging each pose. "Overall" denotes pooling across components, while maximum absolute error is the largest absolute error among all components and samples. These metrics are therefore computed neither from the error-vector norm nor from acceleration-norm deviation relative to gravity.

Table 5.5: Network errors relative to analytical calibration on the test set, in $\mathrm{m/s^2}$.

Component	MAE	RMSE	Maximum absolute
x	4.075×10^{-3}	4.951×10^{-3}	1.118×10^{-2}
y	3.261×10^{-3}	3.650×10^{-3}	1.901×10^{-2}
z	4.655×10^{-3}	6.037×10^{-3}	2.384×10^{-2}
Overall	3.997×10^{-3}	4.976×10^{-3}	2.384×10^{-2}

Overall RMSE is $4.976 \times 10^{-3}\,\mathrm{m/s^2}$, compared with $4.100 \times 10^{-3}\,\mathrm{m/s^2}$ on validation. The z component has both the highest RMSE and the highest maximum absolute error. Figure 5.4 shows variation in overall RMSE between poses, with the largest discrepancy in V09 and the smallest in V01. The aggregate result therefore describes average model behaviour but does not imply uniform accuracy across orientations.

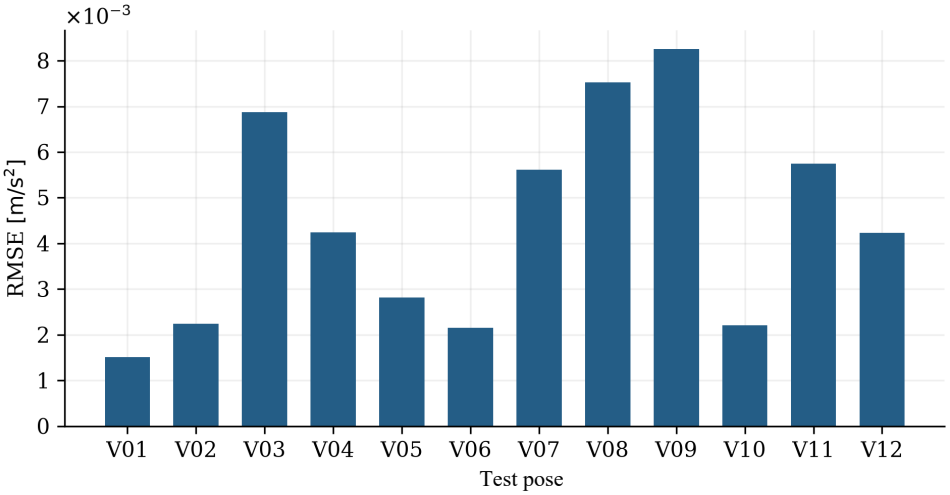


Figure 5.4: Overall network RMSE relative to analytical calibration across the twelve test poses.

To distinguish vector differences from magnitude differences, the quantities $v_i = \|\boldsymbol{e}_i\|_2$ and $d_i = \|\hat{\boldsymbol{a}}_i\|_2 - \|\boldsymbol{a}_i\|_2$ were also considered. Mean vector discrepancy is $7.632 \times 10^{-3}\,\mathrm{m/s^2}$, with a maximum of $2.863 \times 10^{-2}\,\mathrm{m/s^2}$. For the difference between magnitudes, MAE is $4.808 \times 10^{-3}\,\mathrm{m/s^2}$ and RMSE is $5.844 \times 10^{-3}\,\mathrm{m/s^2}$. These latter metrics measure agreement between output norms, not their deviation from gravitational acceleration. Overall, the test quantifies the network's ability to reproduce the analytical transformation on poses excluded from selection. Because calibration itself supplies the reference, it does not independently verify the physical accuracy of individual components. The selected model and training-derived standardisation coefficients are then transferred to the microcontroller. The next chapter describes conversion, numerical output verification and experimental comparison on new acquisitions.

Chapter 6

Embedded Implementation and Experimental Verification

The model selected in Chapter 5, trained with 500 samples per pose, was transferred to the microcontroller to verify that the learned correction could run online, directly during acquisition. Tests use the $3 \rightarrow 10 \rightarrow 3$ network with 73 parameters and the weights saved at epoch 269, together with the same training-derived standardisation coefficients.

6.1 Export and Firmware Integration

Before deployment, four possible solutions were examined: STM32Cube AI Studio, TensorFlow Lite for Microcontrollers (TFLM), CMSIS-NN and the Edge Impulse C++ Inference SDK. Considerations included each tool's role, the transition from trained model to executable code and integration with the chosen microcontroller. STM32Cube AI Studio combines model import, code generation and verification within the STM32 environment [29]. TFLM, also presented in Google documentation as LiteRT for Microcontrollers, is a C++ library for executing models converted to TensorFlow Lite format [30]. CMSIS-NN provides kernels, i.e. optimised functions for neural network computations on Arm Cortex-M processors [31]. Edge Impulse offers a portable C++ SDK containing processing and inference blocks for models exported from its platform, together with a specific STM32CubeMX and STM32CubeIDE integration procedure [33]. Table 6.1 summarises aspects relevant to this project.

Table 6.1: Preliminary comparison of microcontroller deployment tools.

Solution	Function	Integration
STM32Cube AI Studio	Model conversion, optimisation and verification	C code generation for STM32; computer and board verification.
TFLM	C++ microcontroller inference library	TensorFlow Lite conversion; model embedded as a byte array and library integrated into firmware.
CMSIS-NN	Optimised Arm Cortex-M kernels	Functions used by a runtime or application code; not a complete conversion and deployment workflow by itself.
Edge Impulse C++ SDK	Signal processing and inference library	Export as a C++ package; STM32 integration through CMSIS-Pack is also available.

These tools serve different roles and are not all directly equivalent alternatives; for example, CMSIS-NN kernels may form part of another solution's inference system. STM32Cube AI Studio was selected for this work, supplied by STMicroelectronics, the manufacturer of both microcontroller and board. The choice was motivated by straightforward integration with the STM32 tools already in use and the ability to import the model, automatically generate optimised C code and verify outputs on both computer and device within one environment. Selection was based solely on the development workflow; no comparative memory or execution-time measurements were made across the four solutions.

The Keras model was first exported as a TensorFlow SavedModel and then converted with `tf2onnx`, which generates ONNX representations from TensorFlow models [26]. The script specifies opset 13, identifying the ONNX operator-set version. The ONNX model was imported into STM32Cube AI Studio 1.3.0 to generate optimised, board-compatible code using ST Edge AI Core 4.0.1 with *balanced* optimisation and the `st-ai` C interface. This tool generates model code executable through the STMicroelectronics inference library [27]. The network retained 32-bit floating-point representation without integer quantisation, preserving all 73 trained parameters and three inputs and outputs per inference. Generated code, weights and the required runtime library were integrated into the existing STM32CubeIDE sensor-acquisition project. SPI and serial communication retain the configuration in Chapter 3.

The firmware initialises the network before entering the acquisition loop. It first initialises the inference library and model context, then binds the activation, input and output buffers. The context holds information required to manage network execution, while the buffers contain inference data. All operations use the `st-ai` interface documented by STMicroelectronics [28]. For each valid sensor reading, the three counts are converted to floating point and standardised with the training means and standard deviations according to Equation 5.3. Standardisation is implemented in the application code before the network call and is not included in the ONNX model. Inference is synchronous: the program waits for computation to finish before using the three predicted components.

For experimental comparison and performance assessment, the firmware also applies analytical calibration to the same count triplet. A single sensor measurement therefore feeds both calculations, avoiding separate acquisitions for the network and reference. Both outputs internally retain the g_n-normalised representation from Chapter 5; conversion to m/s^2 takes place during processing of the reported results. Supervised-network comparison uses the first eleven serial-line fields: three counts, three analytically calibrated components, three network outputs, and two discrepancy indicators, the maximum and mean absolute differences over the three axes. Data are transmitted through USART3 and acquired in Python for storage and analysis. The computer thus collects results, while analytical calibration and supervised inference execute directly on the microcontroller with fixed parameters.

6.2 Numerical Verification of Deployment

Numerical verification checks that transferring the network between execution environments preserves its outputs. All 18 000 samples from V01–V12 were used, standardising counts with the training coefficients and computing predictions from the selected Keras

model as expected outputs. This checks implementation consistency for fixed inputs and involves neither weight updates nor model selection.

The first comparison was between Keras and ONNX Runtime using the exported model. Next, the *on desktop* and *on target* verification modes in STM32Cube AI Studio were used [29]. These execute network C code on the computer and directly on the NUCLEO-H743ZI2 respectively. Both receive the same 18 000 pre-standardised inputs and Keras reference outputs. This isolates network behaviour from sensor reading and input preparation in the application firmware. Table 6.2 reports MAE, RMSE and maximum absolute error relative to Keras over 54 000 output components. Differences originally in normalised model units were multiplied by g_n to express them in $\mathrm{m/s^2}$.

Table 6.2: Discrepancies relative to Keras outputs over the 18 000 *verification inputs, in* $\mathrm{m/s^2}$.

Execution	MAE	RMSE	Maximum absolute
ONNX Runtime	3.239×10^{-7}	4.663×10^{-7}	2.338×10^{-6}
C code on computer	2.988×10^{-7}	4.307×10^{-7}	2.338×10^{-6}
C code on STM32	3.007×10^{-7}	4.383×10^{-7}	2.338×10^{-6}

The three implementations produce numerically close results. Maximum observed board discrepancy is $2.338 \times 10^{-6}\,\mathrm{m/s^2}$, confirming preservation of outputs for the examined vectors within small numerical differences between environments. These are much smaller than network errors relative to analytical calibration, whose offline component RMSE is $4.976 \times 10^{-3}\,\mathrm{m/s^2}$. The comparisons nevertheless have distinct references and purposes: deployment verification uses Keras predictions, whereas offline comparison with analytical calibration measures approximation of the transformation.

STM32Cube AI generation and execution reports also describe network resource requirements, listed in Table 6.3. The 73 parameters in 32-bit floating point occupy 292 B. The activation buffer requires 52 B and, in the configuration analysed by the tool, includes inputs and outputs through memory reuse during computation. These figures are not the complete firmware Flash and RAM footprint: code, the inference library, network context and other application structures must also be included. In particular, the application declares two additional three-element `float` arrays for input and output.

Table 6.3: Network resources and performance in the STM32Cube AI Studio board test.

Quantity	Value
Network parameters	73, 32-bit floating point
Weight memory	292 B
Activation buffer	52 B, including inputs and outputs
Test inputs	18 000 standardised triplets
Tool-reported operations	173 MACC per inference
Test CPU frequency	480 MHz
Mean cycles per inference	Approximately 2 824
Mean time per inference	Approximately 0.006 ms

The mean inference time of approximately 0.006 ms was measured on target at 480 MHz with instruction and data caches enabled. It concerns network execution on prepared

data and excludes the complete acquisition, calibration and serial-transmission cycle. The sensor application instead uses a 32 MHz system clock, so the verification program's timing cannot be attributed directly to that configuration. Subsequent tests examine the integrated system by comparing network and analytical calibration outputs on new acquisitions.

Figure 6.1 shows the cycle distribution across the model's three layers. The tanh activation accounts for approximately 64.8% of the total, while the linear transformations before and after it account for 18.9% and 16.3% respectively. In this configuration, activation evaluation thus dominates inference time, exceeding the combined contribution of the two layers that perform weight multiplications and bias additions. This breakdown identifies the operations with the greatest execution cost, but concerns only the model in the test configuration and excludes other application tasks such as sensor acquisition and transmission to the PC.

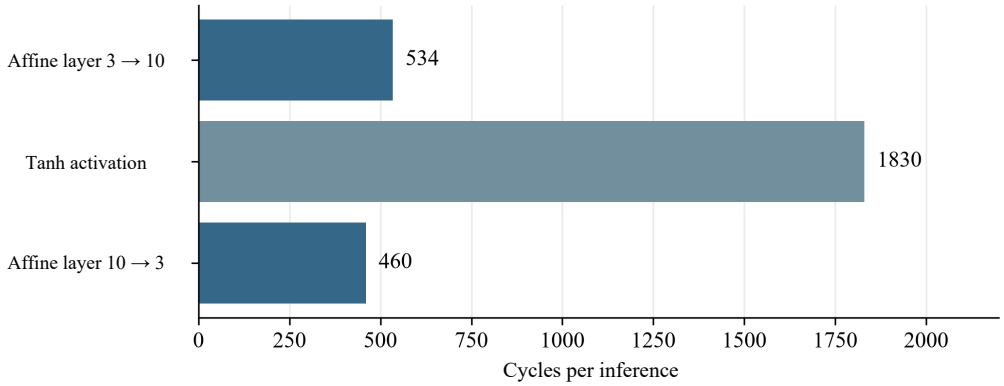


Figure 6.1: Per-layer cycles during on-target verification at 480 MHz, from the STM32Cube AI Studio report.

6.3 Comparison on New Acquisitions

After verification on reference inputs, the system acquired ten new static poses, P01–P10, using a Python script following the same logic as Appendix A. The sensor was repositioned with the supports described in Chapter 3, again choosing random orientations without prescribed angles and keeping it stationary throughout each acquisition. The script waits an additional five seconds, clears the serial buffer and starts collection. Each pose contains 1 000 valid samples, totalling 10 000 measurements. These acquisitions differ from the V poses used for offline testing and are not used to update the network or calibration parameters.

For each sample, the firmware applies analytical calibration and computes the network prediction from the same count triplet. The comparison therefore concerns the difference between two outputs obtained by processing the same measurement on the microcontroller. Discrepancies were evaluated on individual samples, without prior pose averaging, using the MAE and RMSE definitions in Equation 5.6. Table 6.4 reports per-axis and overall results across the 30 000 recorded components. All errors are in $\mathrm{m/s^2}$, obtained by multiplying normalised differences transmitted by the board by g_n.

Table 6.4: Network errors relative to analytical calibration on new acquisitions, in $\mathrm{m/s^2}$.

Component	MAE	RMSE	Maximum absolute
x	4.473×10^{-3}	4.896×10^{-3}	8.089×10^{-3}
y	2.579×10^{-3}	3.633×10^{-3}	1.546×10^{-2}
z	4.103×10^{-3}	4.802×10^{-3}	1.263×10^{-2}
Overall	3.718×10^{-3}	4.481×10^{-3}	1.546×10^{-2}

Overall RMSE is $4.481 \times 10^{-3}\,\mathrm{m/s^2}$, of the same order as the offline value of $4.976 \times 10^{-3}\,\mathrm{m/s^2}$. Both campaigns show similar overall discrepancies, but their difference does not measure deployment error because the inputs differ. The x component has the highest RMSE, slightly above z. Figure 6.2 shows variation between poses: overall RMSE ranges from approximately $1.729 \times 10^{-3}\,\mathrm{m/s^2}$ in P02 to $7.171 \times 10^{-3}\,\mathrm{m/s^2}$ in P08.

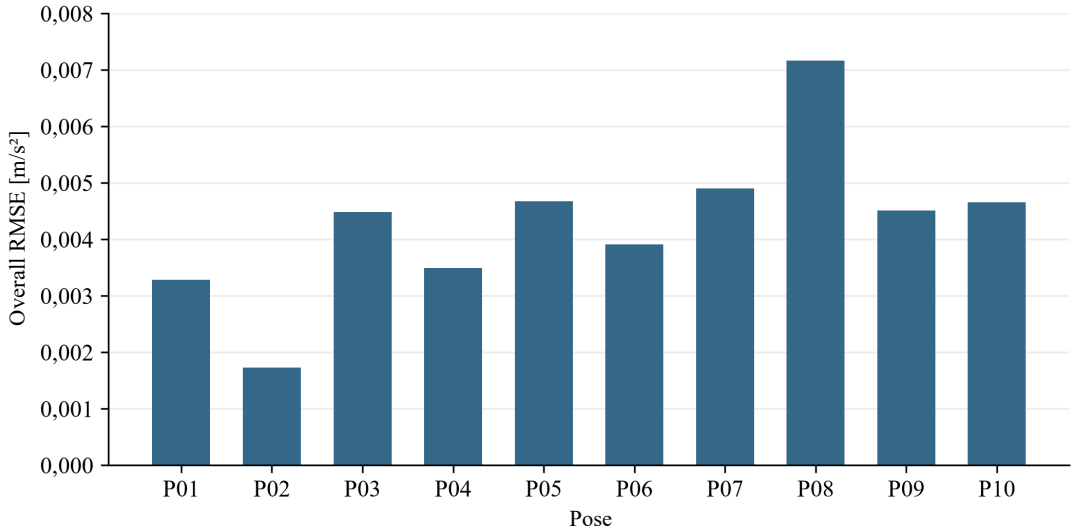


Figure 6.2: Overall RMSE between network and analytical calibration for the ten new static poses.

The maximum recorded absolute error is $1.546 \times 10^{-2}\,\mathrm{m/s^2}$, affecting the y component of sample 305 in P05. All measurements were retained in the metrics, including the largest discrepancies. P05 shows a brief count variation around the largest discrepancy, while the remaining measurements stay close to the mean; the available data do not identify a specific experimental cause. Received outputs were also checked against computer recalculations from the same counts, giving a maximum network discrepancy of approximately $2.341 \times 10^{-6}\,\mathrm{m/s^2}$. This check includes firmware input preparation and supports consistency of the computational chain on the acquired measurements.

Overall, these tests document network execution within the acquisition system and quantify agreement with the analytical reference at additional orientations. Although the poses are distinct, they do not uniformly cover orientation space, so results remain specific to the examined conditions. Evaluation against the gravitational constraint and comparison with nominal Factory conversion follow.

6.4 Performance Discussion

Comparison with analytical calibration quantifies how faithfully the network reproduces its outputs and therefore how well it performs its intended task. To also assess behaviour against the physical calibration constraint, Factory, analytical calibration and the supervised network were compared using acceleration norms. This analysis uses the twelve offline test poses V01–V12, distinct from P01–P10 above. For each method and pose, output vectors are averaged before calculating the mean vector's norm. Deviation from the reference is

$$d_{p,m} = \left\| \overline{\boldsymbol{a}}_{p,m} \right\|_2 - g_n, \qquad m \in \{\text{Factory}, \text{Cal}, \text{NN}\}, \tag{6.1}$$

where p identifies the pose and m the method. Consistently with Section 4.3, metrics use the twelve pose-mean deviations rather than individual sample norms. For the network, the mean is calculated from sample predictions, not replaced by a prediction at the mean input.

Table 6.5: Deviation of pose-mean norms from the gravitational reference, in $\mathrm{m/s^2}$.

Metric	Factory	Calibration	Supervised network
RMSE	0.059047	0.002028	0.006681
MAE	0.047379	0.001763	0.005451
Maximum absolute	0.098703	0.003330	0.012240

Table 6.5 shows that both methods reduce aggregate deviations relative to Factory conversion. The network achieves an RMSE of $0.006681\,\mathrm{m/s^2}$, an 88.68% reduction; MAE and maximum absolute error decrease by 88.50% and 87.60% respectively. Analytical calibration retains lower values for all three aggregate metrics. As with analytical calibration, improvement is not uniform across poses: Factory deviation is already close to zero in V04 and the network increases its absolute value, whereas in other poses, such as V06, the network is even closer to the reference than analytical calibration. This local result does not demonstrate greater general network accuracy, since approximation error can partly compensate for the analytical reference's deviation. Overall, however, much of the norm improvement achieved by calibration is transferred to the network.

Direction was examined separately by comparing network and analytical calibration mean vectors over the same twelve poses. The latter provide the reference because they are the outputs the network was trained to reproduce. Angular deviation is

$$\theta_p = \arccos\left(\frac{\overline{\boldsymbol{a}}_{p,\text{NN}}^{\mathsf{T}} \overline{\boldsymbol{a}}_{p,\text{Cal}}}{\left\| \overline{\boldsymbol{a}}_{p,\text{NN}} \right\|_2 \left\| \overline{\boldsymbol{a}}_{p,\text{Cal}} \right\|_2} \right). \tag{6.2}$$

In degrees, the deviations have a mean of 0.0313°, a root mean square of 0.0370° and a maximum of 0.0702° in V08. Network direction therefore remains close to that of the calibrated vector for the examined poses. However, the reference remains the analytical model, and, as discussed in Section 4.4, the norm constraint alone does not determine vector orientation. These values measure agreement between methods, not error relative to an independently measured direction.

The checks document three complementary aspects: numerical preservation of outputs during deployment, operation on new acquisitions, and reduced reference-norm deviations relative to Factory. For this problem, the network is an embedded approximation

of analytical calibration, which retains the best aggregate norm metrics. No execution-time or memory advantage over analytical computation was measured. Conclusions also remain limited to the tested static conditions and do not extend to dynamic measurements or different environments. Adaptation to new acquisitions through continual parameter updates is addressed next.

Chapter 7

Unsupervised Learning and Continual Learning

7.1 Principles and Applications of Continual Learning

In the preceding chapters, the neural network was trained on a defined dataset and then transferred to the microcontroller, where its parameters remained unchanged throughout measurement processing. Such a model may perform correctly on acquisitions excluded from development, but performance can deteriorate when operating conditions differ from those represented in training data. How to update it when new measurements become available therefore remains unresolved. *Continual learning* addresses this problem through methods that learn from a sequence of experiences while retaining previously acquired capabilities. Data become available progressively; at each update, earlier data may no longer be accessible or may be retained only partially. New information may consist of additional examples of the same phenomenon, different operating conditions or additional tasks, so continual learning does not necessarily require a change in the system's function [34].

This approach is attractive because a model can be updated without repeating full training on all accumulated data, which may become costly and time-consuming as acquisition continues. The aim is to use new observations with limited memory and computational requirements while retaining information still useful for the task. "Continual" describes the progressive nature of learning and does not imply uninterrupted or per-sample updates: depending on the procedure, data may be processed individually or in successive groups. Likewise, new measurements do not necessarily come with correct reference outputs, motivating approaches with limited or no supervision [35].

The main difficulty is that parameter updates may improve performance on new data while degrading it on previous experiences. In neural networks, this is known as *catastrophic forgetting*: weights modified to learn new information also contribute to previously learned responses. A trade-off therefore arises between plasticity, the ability to adapt, and stability, needed to retain acquired knowledge. One proposed strategy is parameter regularisation, penalising changes to weights considered important for earlier tasks. An example is *Elastic Weight Consolidation*, studied by Kirkpatrick and colleagues [36] for classification and reinforcement learning.

Other strategies reuse some previous data during updates through *replay*, or modify the architecture to limit interference between knowledge acquired at different times. These solutions have different costs: retaining examples requires memory, while expanding the model can increase complexity. Evaluation must therefore assess both acquisition of new information and retention of previous performance, together with the resources

needed. Reducing error only on recently acquired data is insufficient to establish overall effectiveness [35].

Robotic vision is one field where these issues are studied: a system may progressively encounter objects and viewing conditions not fully represented in its initial data. Recognition must be updated without losing the ability to identify familiar objects or rebuilding the entire model for each new acquisition. An experimental reference is CORe50, introduced by Lomonaco and Maltoni [37] to compare continual object-recognition methods through learning sequences. This example also highlights the importance of common evaluation protocols for examining model behaviour throughout updates and comparing strategies under identical conditions.

In mechanical engineering, continual learning is investigated for fault diagnosis, where signal distributions may change with operating conditions and previously unconsidered anomalies may become available. Li and colleagues [38] addressed these issues by combining continual learning and transfer between conditions, evaluating their method on an automotive transmission dataset. The aim was to acquire new diagnostic information while preserving earlier knowledge, keeping the model useful as available data evolved. This is an experimental machine-monitoring application that suggests potential value for data-driven maintenance, rather than evidence of widespread industrial deployment.

Another research direction concerns updating models directly on data-acquisition devices, adapting them to local observations while reducing, or sometimes eliminating, transfer to an external computer. This requires more resources than inference alone, since training must compute parameter changes and retain intermediate information. Lin and colleagues [39] demonstrated adaptation of pretrained convolutional networks for visual recognition on a microcontroller with 256 kB SRAM, using specific memory-reduction techniques. This establishes the feasibility of local training under constrained conditions but is not a general solution to forgetting, which requires separate evaluation.

For future industrial systems, this research suggests models that can be updated throughout machine operation, incorporating new observations while retaining performance under familiar conditions. Prospective applications include adaptive diagnostics and intelligent devices that learn locally. Their implementation nevertheless requires representative sequential-data evaluation and compliance with resource limits. Studies of continual learning on embedded devices highlight the importance of considering performance, memory and computational cost together, even when data arrive one sample at a time [40]. Within this context, the final part of this thesis updates an unsupervised network trained with the gravity constraint in a Python simulation to study adaptation to a controlled change in accelerometer behaviour.

7.2 Unsupervised Pretraining with the Norm Constraint

The previous chapters examined whether a neural network could learn the analytical calibration transformation and reproduce it on a microcontroller. Building on that result, a different approach was considered in which calibrated outputs are no longer provided as targets. A separate network, termed the unsupervised network below, was pretrained using the physical constraint that the ideal static measurement norm

equals gravitational acceleration. A directional regularisation term limits deviation from Factory measurement directions, restraining unwanted rotations without introducing a calibrated three-component reference. The resulting weights initialise the subsequent adaptation simulation.

Twenty poses, C07–C26, supplied 30 000 samples. Static poses 41–50, comprising 5 000 samples, were reserved for validation. The six axial poses C01–C06 were excluded from updates in this phase and used to examine model-induced directional deviation. Inputs were formed by dividing sensor counts by the nominal 6000 counts corresponding to g_n, yielding dimensionless Factory measurements. For Factory vector $\mathbf{a}_{\mathrm{F},i}$ in $\mathrm{m/s^2}$,

$$\mathbf{x}_i = \frac{\mathbf{a}_{\mathrm{F},i}}{g_n}, \qquad g_n = 9.80665\ \mathrm{m/s^2}. \tag{7.1}$$

The static constraint is therefore a unit-norm condition, and network outputs are converted to physical units by $\widehat{\mathbf{a}}_i = g_n \widehat{\mathbf{y}}_i$.

The model was implemented in Python with PyTorch, using three inputs, one hidden layer of ten hyperbolic-tangent neurons and three linear outputs. Input-to-hidden and hidden-to-output transformations use two `Linear` modules containing weight matrices and bias vectors [41]. Unlike the supervised network, a residual connection adds the input to the network's predicted correction, following the residual learning principle described by He et al. [42]. This starts from Factory measurements, already internally compensated, and lets the neural branch learn an additional correction without analytical calibration targets. The output is

$$\widehat{\mathbf{y}}_i = \mathbf{x}_i + \mathbf{W}_2 \tanh(\mathbf{W}_1 \mathbf{x}_i + \mathbf{b}_1) + \mathbf{b}_2, \tag{7.2}$$

where $\mathbf{W}_1 \in \mathbb{R}^{10\times 3}$ and $\mathbf{W}_2 \in \mathbb{R}^{3\times 10}$ are weight matrices, and $\mathbf{b}_1 \in \mathbb{R}^{10}$ and $\mathbf{b}_2 \in \mathbb{R}^3$ are bias vectors; hyperbolic tangent is applied componentwise. The model retains 73 trainable parameters because input addition introduces no coefficients. Final-layer weights and biases were initialised to zero so that the initial output equalled the input and training progressively learned a Factory correction.

The loss comprises a norm term and directional regularisation. For N samples, the former is

$$\mathscr{L}_{\mathrm{norm}} = \frac{1}{N} \sum_{i=1}^{N} \left(\left\| \widehat{\mathbf{y}}_i \right\|_2 - 1 \right)^2, \tag{7.3}$$

The latter penalises deviation between output direction and Factory input direction:

$$\mathscr{L}_{\mathrm{dir}} = \frac{1}{N} \sum_{i=1}^{N} \left(1 - \frac{\widehat{\mathbf{y}}_i^{\mathsf{T}} \mathbf{x}_i}{\left\| \widehat{\mathbf{y}}_i \right\|_2 \left\| \mathbf{x}_i \right\|_2} \right). \tag{7.4}$$

For non-zero vectors, the ratio is the cosine of their angle, so the directional term vanishes when the directions coincide. The dimensionless total loss is

$$\mathscr{L}_{\mathrm{pre}} = \mathscr{L}_{\mathrm{norm}} + \lambda_{\mathrm{dir}}(e)\, \mathscr{L}_{\mathrm{dir}}, \qquad \lambda_{\mathrm{dir}}(e) = 0.1 \min\!\left(1, \frac{e}{200}\right), \tag{7.5}$$

where e is the epoch index starting at one. The coefficient therefore increases according to the specified ramp. Directional regularisation limits changes relative to the initial measurement without assuming that Factory direction is exact. The norm constraint

alone cannot distinguish vectors of equal magnitude with different orientations; regularisation favours corrections preserving the initial direction. Numerically, norms are bounded below by 10^{-8} and the cosine is clipped to $[-1, 1]$.

Training uses Adam [20], reshuffling data each epoch with the settings in Table 7.1. The model is evaluated on the full validation set without using those samples for weight updates. Training stops when total validation loss fails to improve by more than the specified threshold for the stated number of consecutive epochs, retaining the checkpoint with the best recorded value.

Table 7.1: Residual network pretraining settings.

Parameter	Setting
Architecture	Residual 3–10–3, 73 parameters
Hidden activation / branch output	Tanh / linear
Optimiser	Adam
Learning rate	10^{-3}, constant
Mini-batch size	256 samples
Maximum epochs	1200
Early-stopping patience	100 epochs
Improvement threshold	10^{-12} on validation loss
Directional coefficient	Ramp in Eq. (7.5)
Seed	42

In the run performed, the selected checkpoint is from epoch 1 and training stops at epoch 101. Since the directional coefficient changes during training, total-loss comparisons are also affected by that change. Nevertheless, the available logs show that the validation norm term alone also reaches its minimum at epoch 1. This selected model provides the initial reference for assessing subsequent continual learning updates. Pretraining code is provided in Appendix D.

7.3 Evaluation of the Unsupervised Network

Before sequential updates, the unsupervised network was evaluated on V01–V12 and compared with Factory data, analytical calibration and the supervised network. The comparison follows Section 6.4: individual samples are processed, output vectors are averaged per pose, and their mean-vector norms are compared with g_n. Metrics thus use twelve pose deviations and are distinct from individual-sample norm evaluation. For the pretrained model, normalised outputs were averaged in double precision and the resulting vector converted to $\mathrm{m/s^2}$. Table 7.2 retains the Factory and analytical calibration values from Table 6.5; supervised results refer to the same model described in Chapter 5 and used in embedded verification.

The pretrained network reduces all three metrics relative to Factory data: RMSE by 93.64%, MAE by 93.88% and maximum absolute error by 91.71%. On these poses, pretraining therefore corrects a substantial fraction of norm error without calibrated

Table 7.2: Deviation of pose-mean norms from the gravitational reference, in $\mathrm{m/s^2}$.

Metric	Factory	Analytical calibration	Supervised network	Unsupervised network
RMSE	0.059047	0.002028	0.006681	0.003757
MAE	0.047379	0.001763	0.005451	0.002901
Maximum absolute	0.098703	0.003330	0.012240	0.008186

component references. The largest unsupervised-network deviation occurs in V11 and is approximately $0.008186\,\mathrm{m/s^2}$.

Interpretation must account for the different losses used by the two networks. The supervised model learns the three analytically calibrated components, whereas pre-training directly uses the norm constraint together with directional regularisation from Section 7.2. Lower error against that constraint does not demonstrate greater accuracy of the entire vector, since differently directed outputs may have equal magnitude. These results characterise the unsupervised network before data modification. Next, a controlled perturbation is introduced to compare the fixed-weight model with one progressively updated under the new sensor condition.

7.4 Simulation of Adaptation to Calibration Drift

Having verified that a supervised network can learn the analytical transformation and an unsupervised network can reduce norm error without calibrated targets, the possibility of updating the latter when sensor behaviour changes was examined. Previously acquired measurements were perturbed to simulate changes in scale factors and offsets, two accelerometer error sources [5]. For original Factory measurement $\mathbf{a}_{\mathrm{F},i}$, the altered measurement is obtained through the affine transformation

$$\widetilde{\mathbf{a}}_{\mathrm{F},i} = \mathbf{G}\mathbf{a}_{\mathrm{F},i} + \mathbf{b}, \qquad \mathbf{G} = 1.10\,\mathbf{I}_3, \qquad \mathbf{b} = \begin{bmatrix} 0.50 \\ 0.50 \\ 0.50 \end{bmatrix} \mathrm{m/s^2}. \tag{7.6}$$

Gain therefore increases by 10%, and $0.50\,\mathrm{m/s^2}$ is added to each component, with identical coefficients for every pose. These values define a substantial, controlled perturbation chosen to make adaptation observable; they are neither measured device drift nor a prediction of thermal behaviour or ageing. The change is abrupt and remains constant throughout the test. The simulation retains original acquisition variability, scaled by the gain, without adding noise or simulating new quantisation.

Original data were kept separate from the altered copy. Updates used the forty 500-sample static poses already designated as the learning stream. The remaining ten 500-sample static poses monitored error only, while the twelve 1500-sample V poses were reserved for final comparison. The same transformation was applied to all three sets. Before network input, measurements were divided by g_n, giving $\widetilde{\mathbf{x}}_i = \mathbf{G}\mathbf{x}_i + \mathbf{b}/g_n$. Perturbation coefficients are used to generate data but are not supplied to the update algorithm, which receives only altered triplets and the unit-norm constraint. This is

therefore a numerical test based on existing measurements, not new experimental acquisition.

The unsupervised network from Section 7.2 provides the common starting point for both cases: one freezes its weights, and the other updates them progressively as poses are presented one at a time. After each pose, 100 samples are retained without replacement in a replay memory of previously encountered altered observations. Each update draws 128 samples with replacement from the current pose and 128 from memory. Since each pose contributes the same retained sample count, uniform memory sampling assigns equal total probability to each previous pose. For the first pose, when memory is empty, all 256 samples come from the current pose. Memory contains only observations from the new sensor condition and grows to 4000 samples. Original measurements are not used as anchors, since their relationship to physical acceleration predates the perturbation.

For the resulting mini-batch $\mathscr{B}$ and residual network $\boldsymbol{f}_{\boldsymbol{\theta}}$, all 73 parameters are updated by minimising

$$\mathscr{L}_{\mathrm{CL}}(\boldsymbol{\theta}) = \frac{1}{|\mathscr{B}|} \sum_{\tilde{\mathbf{x}}_i \in \mathscr{B}} \left(\left\| \boldsymbol{f}_{\boldsymbol{\theta}}(\tilde{\mathbf{x}}_i) \right\|_2 - 1 \right)^2 . \tag{7.7}$$

The pretraining directional term is omitted here because the introduced offset also changes measurement direction, and preserving it could obstruct correction. Adam is again used, retaining its state between poses and using Table 7.3. The 100 updates per pose are operations on randomly sampled mini-batches, not 100 complete dataset epochs. The final model is the one obtained after pose 40, without early stopping or weight selection using the V poses or monitoring set.

Table 7.3: Update settings under the altered condition.

Parameter	Setting
Initial model	Unsupervised network, 73 parameters
Optimiser / learning rate	Adam / 10^{-3}, constant
Poses / samples per pose	40 / 500
Updates per pose / total	100 / 4000
Current / replay mini-batch	128 / 128 samples
Memory per pose / final	100 / 4000 samples
Weights used in final comparison	After pose 40
Sampling seeds	42, 43, 44, 45, 46

Settings were fixed before execution without a test-set parameter search. Five sampling seeds were used, keeping the pretrained model and pose order unchanged. Seed 42 provides the main comparison; the other four characterise variability due to sample draws. V poses participate in neither updates nor configuration selection for this test. They nevertheless were already examined in earlier comparisons, so their reuse does not constitute a new independent experimental campaign. Update code is provided in Appendix E.

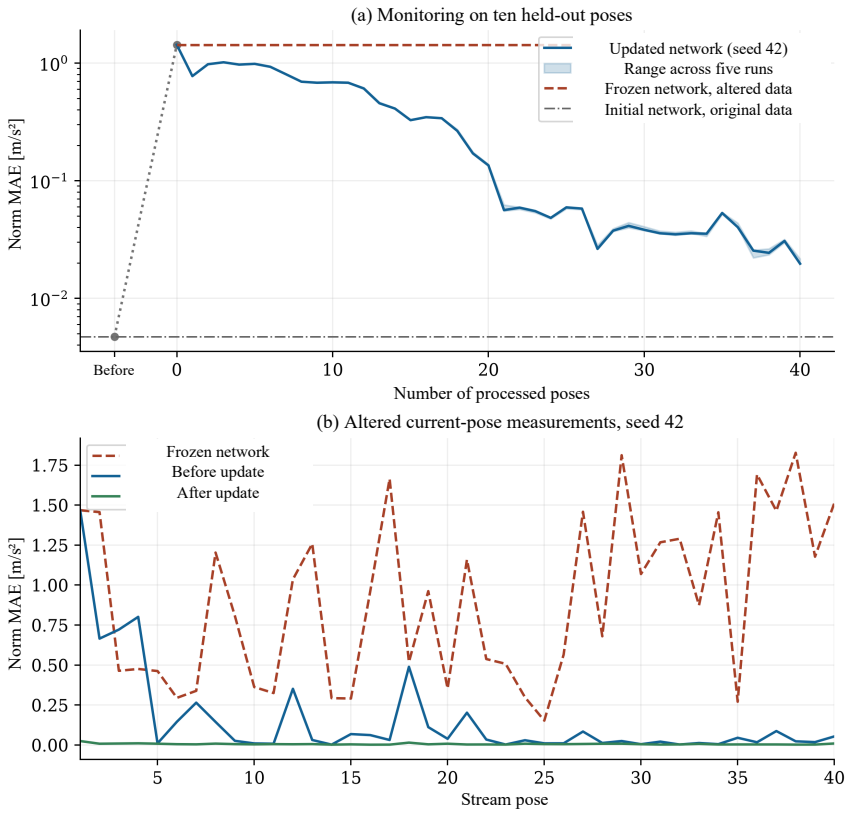


Figure 7.1: Error on monitoring poses and the current pose during adaptation.

7.5 Adaptation Results and Model Comparison

The perturbation's effect was examined on the ten monitoring poses using individual-sample norm MAE in m/s^2. As Figure 7.1 shows, the initial network's error is approximately $0.004710\,m/s^2$ on original data, rising to $1.427936\,m/s^2$ when gain and bias are introduced. After updates, the main run reaches $0.019651\,m/s^2$, with oscillations during adaptation. The pre-perturbation and initial altered points use identical poses and weights, so the initial increase is attributable solely to changed inputs. The shaded band represents the minimum and maximum of the five runs, not a confidence interval.

For each stream pose, error was measured before and after its 100 updates, retaining the frozen network for comparison. The immediate reduction measures adaptation to the just-used pose, not test performance. The pre-update curve shows response before that pose contributes to gradients; error on newly learned data alone does not measure generalisation.

Final comparison on the twelve altered V poses follows Section 7.3: output vectors are averaged per pose and their norms compared with g_n. Table 7.4 includes the original condition to quantify recovery relative to pre-perturbation performance. In the main run, RMSE falls from 1.066069 to $0.019333\,m/s^2$, a 98.19% reduction relative to the frozen network on the same altered data. All twelve poses improve, as Figure 7.2 shows, but error remains above the initial network's error on original data. Adaptation therefore compensates for much of the introduced effect without fully restoring initial performance.

Across five runs, pose-mean RMSE ranges from 0.019170 to $0.020061\,m/s^2$, with mean $0.019487\,m/s^2$ and sample standard deviation $0.000342\,m/s^2$. This indicates limited

Table 7.4: Norm errors of V-pose mean vectors, in $\mathrm{m/s^2}$.

Condition	RMSE	MAE	Maximum
Initial network, original data	0.003757	0.002901	0.008186
Initial network, altered data	1.066069	0.932010	1.822832
Updated network, altered data	0.019333	0.015341	0.042036

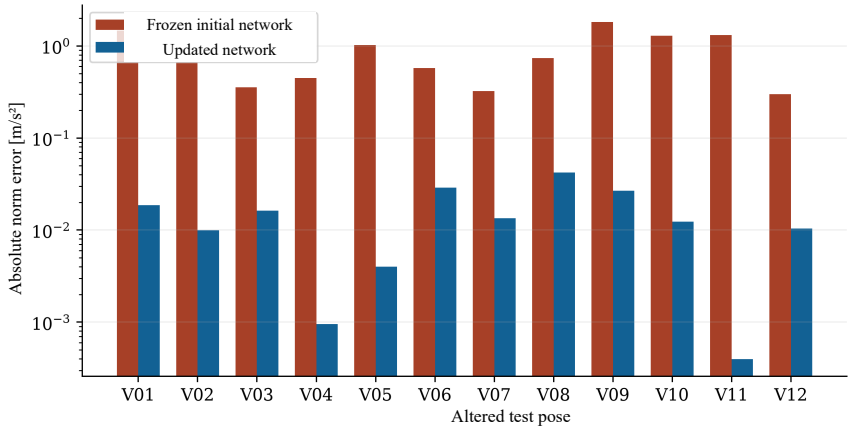


Figure 7.2: Pose-mean norm error on the altered test set, with a logarithmic vertical scale.

sensitivity to sample draws, but not to pose order, pretraining or perturbation, which remained fixed.

These pose-mean metrics differ from individual-sample errors: the main run's norm RMSE over 18000 test samples is $0.022159\,\mathrm{m/s^2}$, compared with $0.019333\,\mathrm{m/s^2}$ for the twelve mean vectors.

Since the norm does not uniquely determine the components, outputs were also compared with the initial network on corresponding original measurements, $\mathbf{a}_{0,i} = g_n \boldsymbol{f}_{\boldsymbol{\theta}_0}(\mathbf{x}_i)$. Component recovery RMSE is

$$E_{\mathrm{rec}} = \sqrt{\frac{1}{3N}\sum_{i=1}^{N} \left\| g_n \boldsymbol{f}_{\boldsymbol{\theta}}(\tilde{\mathbf{x}}_i) - \mathbf{a}_{0,i} \right\|_2^2}. \tag{7.8}$$

On V-pose samples, this metric falls from 0.723826 to $0.144162\,\mathrm{m/s^2}$, and the mean angle between pose mean vectors and their references from 3.5421° to 1.3582°. Used only for this check, never as an update target, the reference describes previous network behaviour, not independently measured acceleration. Component recovery is therefore partial, with directional discrepancies unresolved by the norm constraint.

Adaptation is specific to the new condition: on the ten unaltered monitoring poses, the final model's sample norm MAE rises to approximately $1.290297\,\mathrm{m/s^2}$. It neither retains both corrections nor demonstrates absence of forgetting. Physically constrained sequential learning reduces error after the controlled affine perturbation without analytical targets. Other perturbations, actual sensor changes and strategies for recognising and managing multiple conditions require further study. Updates were simulated in Python; microcontroller weight updates remain future work.

Chapter 8

Conclusions and Future Work

8.1 Results and Concluding Remarks

This work investigated calibration of a Murata SCL3300 triaxial accelerometer through a process beginning with measurement acquisition and analytical transformation identification, followed by neural network development and deployment on an STM32 NUCLEO-H743ZI2 board. The objective was to determine whether a compact model could learn the calibration correction and reproduce it directly on the microcontroller. This was followed by a separate unsupervised network, initially trained with a physical constraint and subsequently updated in a Python continual learning simulation.

Analytical calibration, identified from the twenty-six static C poses, reduced deviations in the measurement norm from the conventional gravitational reference. On the twelve V poses excluded from identification, pose-mean norm RMSE decreased from approximately $0.05905\,\mathrm{m/s^2}$ for Factory measurements to $0.00203\,\mathrm{m/s^2}$ after calibration. This confirms effectiveness under the examined static conditions. Directional tests, however, remain affected by the lack of a reliable independent angular reference, preventing the observed deviations from being interpreted as absolute accuracy.

The supervised comparison used 500 samples per pose with training and validation acquisitions kept separate and selected the 73-parameter 3–10–3 network. Its offline overall component MAE relative to calibrated outputs is approximately $3.997\times10^{-3}\,\mathrm{m/s^2}$. ONNX conversion and STM32Cube AI Studio enabled firmware integration while preserving outputs within numerical discrepancies of order $10^{-6}\,\mathrm{m/s^2}$ relative to Keras. On ten additional poses acquired with the complete system, overall MAE and RMSE relative to analytical calibration are 3.718×10^{-3} and $4.481\times10^{-3}\,\mathrm{m/s^2}$ respectively. The 73 parameters occupy 292 bytes in 32-bit format, which is not the total memory requirement. Experiments therefore show that a small network can reproduce a calibration-derived correction on the device with good agreement with model outputs. However, no advantage over direct use of the analytical model emerges; it continues to provide the lowest V-pose norm error. Once parameters are estimated, analytical calibration itself is an affine transformation executable on a microcontroller. The supervised stage thus demonstrates that a small network can learn an approximation to that transformation and reproduce it on an embedded platform. Establishing execution-time, memory or energy benefits would require a targeted comparison of both implementations.

The unsupervised stage pursued a different objective: training without analytical calibration targets, using the norm constraint and directional regularisation relative to Factory measurements. Starting from this model, a sensor-behaviour change was simulated by applying a gain of 1.10 and an offset of $0.50\,\mathrm{m/s^2}$ to all components, then updating the network through forty altered static poses. On the twelve V poses under the same

perturbation, pose-mean norm RMSE decreased from approximately 1.06607 m/s^2 for the frozen model to 0.01933 m/s^2 after adaptation, a 98.19% reduction. These poses were not used to select adaptation parameters or weights, although they belong to a campaign examined previously. Recovery remains partial relative to the original condition, component checks reveal residual directional discrepancies, and performance worsens when the updated model is applied again to unaltered measurements. The result therefore documents adaptation to the simulated perturbation without demonstrating complete vector calibration or simultaneous retention across conditions. This procedure was executed in Python and remains distinct from supervised embedded inference.

8.2 Future Work

Experimental verification could be extended with new poses reserved entirely for final evaluation and an independent angular reference to separate norm and direction errors. Repeated tests on multiple sensors at controlled temperatures within datasheet limits would assess dependence on the device and acquisition conditions, and whether temperature should become a model input.

The five continual learning runs varied sample draws only, keeping the initial model fixed. Further tests should vary pose order, pretraining, and perturbation magnitude, sign and time history. Comparison without replay would quantify its contribution; fixed-capacity memory would suit longer sequences. Detecting sensor-condition changes and returns to earlier conditions also requires investigation, since the present method does not retain both corrections. Operation would require detection of stationary intervals: the gravitational constraint cannot be applied indiscriminately during motion-induced acceleration.

Embedded weight updates require verification of execution time, energy use and memory for data, gradients and optimiser state. A small network does not guarantee that training meets device constraints. Lin and colleagues [39] demonstrated training with 256 kB SRAM using specific techniques; Hayes and Kanan [40] examined both performance and resource requirements for embedded continual learning. These studies inform future implementation, including criteria for accepting updates or restoring earlier weights when performance deteriorates.

An updatable measurement system could maintain reliable data for machine control and monitoring as operating conditions change. Outside calibration, Li and colleagues [38] investigated continual learning for automotive-transmission fault diagnosis under non-stationary conditions. Connecting the present procedure to such applications requires demonstrating benefits for the final measurement or system decision through dynamic tests with suitable references. This work provides an experimental foundation: supervised embedded execution was validated, while physically constrained updates require independent tests and verification on the device.

Appendix A

Pose Acquisition Code

The listing provides the Python code for acquiring a static pose, shared by the three campaigns in Section 3.4. The script `acquire_pose.py` waits five seconds and clears the buffer, then reads digital-count triplets from the serial port until `NUM_SAMPLES` valid samples have been collected. It saves a CSV file with columns `sample`, `raw_x`, `raw_y` and `raw_z`. `POSE_NAME` defines the filename and `OUTPUT_DIRECTORY` the destination folder; changing campaigns requires adjusting these parameters as in Table A.1.

Table A.1: Script parameters for the three acquisition campaigns.

Campaign	Samples per pose	Pose name
Calibration	1500	`C01–C26`
Additional poses	500	`static_001–static_050`
External test	1500	`V01–V12`

Each execution acquires one pose, during which the sensor must remain stationary. Before the next acquisition, change the pose name and reposition the sensor. The destination folder, represented by `Path`, is selected separately for each campaign. Port `COM5` and 115200 baud match the experimental configuration. The program requires pySerial and is run with `python acquire_pose.py` after closing YAT or other applications using the same port. It exits after saving the samples and does not overwrite existing files.

```
import csv
import time
from pathlib import Path
import serial
SERIAL_PORT = "COM5"
BAUD_RATE = 115200
NUM_SAMPLES = 1500
POSE_NAME = "C01"
OUTPUT_DIRECTORY = Path(__file__).resolve().parent / "Dataset_Calibration"
def main():
    OUTPUT_DIRECTORY.mkdir(parents=True, exist_ok=True)
    output_file = OUTPUT_DIRECTORY / f"{POSE_NAME}.csv"
    if output_file.exists():
        raise FileExistsError(output_file)
    if NUM_SAMPLES < 1:
        raise ValueError("NUM_SAMPLES must be positive")
    samples = []
    with serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1.0) as connection:
        connection.reset_input_buffer()
        time.sleep(5)
        connection.reset_input_buffer()
        while len(samples) < NUM_SAMPLES:
            try:
                fields = connection.readline().decode("utf-8").strip().split(",")
```

```
                if len(fields) != 3:
                    continue
                samples.append(tuple(int(value) for value in fields))
            except (UnicodeDecodeError, ValueError):
                continue
    with output_file.open("x", newline="", encoding="utf-8") as csv_file:
        writer = csv.writer(csv_file)
        writer.writerow(["sample", "raw_x", "raw_y", "raw_z"])
        writer.writerows(
            (index, *sample) for index, sample in enumerate(samples, start=1)
        )
if __name__ == "__main__":
    main()
```

Appendix B

Analytical Calibration Code

The listing provides Python code for analytical accelerometer calibration. It averages the 26 C poses, identifies nine parameters using Levenberg–Marquardt and saves them as JSON, with the settings in Section 4.2. CSV files C01–C26, each containing 1500 samples, are read from `Dataset_Calibration` beside the script. Results are saved to `Calibration_Results/calibration_parameters.json`.

The saved `scale_x`, `scale_y` and `scale_z` coefficients return accelerations normalised by g_n. To obtain the coefficients in $\mathrm{m/s^2}$ per count in Table 4.2, multiply each by $g_n = 9.80665\,\mathrm{m/s^2}$. Non-orthogonality parameters and offset correction terms remain unchanged. The program requires NumPy and SciPy.

```
import json
from pathlib import Path
import numpy as np
from scipy.optimize import least_squares
ROOT = Path(__file__).resolve().parent
DATASET = ROOT / "Dataset_Calibration"
OUTPUT = ROOT / "Calibration_Results" / "calibration_parameters.json"
POSES = [f"C{i:02d}" for i in range(1, 27)]
PARAMETERS = [
    "alpha_yz", "alpha_zy", "alpha_zx",
    "scale_x", "scale_y", "scale_z",
    "bias_x", "bias_y", "bias_z",
]
def load_means(directory):
    means = []
    for pose in POSES:
        data = np.loadtxt(directory / f"{pose}.csv", delimiter=",",
                          skiprows=1, ndmin=2, encoding="utf-8-sig")
        if data.shape != (1500, 4) or not np.all(np.isfinite(data)):
            raise ValueError(f"{pose}: invalid data")
        means.append(data[:, 1:].mean(axis=0))
    return np.asarray(means)
def initial_parameters(means):
    positive = means[[0, 2, 4], [0, 1, 2]]
    negative = means[[1, 3, 5], [0, 1, 2]]
    span = positive - negative
    if np.any(span <= 0):
        raise ValueError("Inconsistent axial pose orientations")
    scales = 2.0 / span
    biases = -(positive + negative) / 2.0
    return np.concatenate((np.zeros(3), scales, biases))
def residuals(parameters, means):
    alpha_yz, alpha_zy, alpha_zx = parameters[:3]
    transform = np.array([
        [1.0, -alpha_yz, alpha_zy],
        [0.0, 1.0, -alpha_zx],
        [0.0, 0.0, 1.0],
    ])
    calibration = transform @ np.diag(parameters[3:6])
```

```
    corrected = (calibration @ (means + parameters[6:]).T).T
    return 1.0 - np.sum(corrected ** 2, axis=1)
def estimate_parameters(means):
    result = least_squares(
        residuals, initial_parameters(means), args=(means,),
        method="lm", jac="2-point", x_scale="jac", loss="linear",
        ftol=1e-12, xtol=1e-12, gtol=1e-12, max_nfev=10000,
    )
    if not result.success or not np.all(np.isfinite(result.x)):
        raise RuntimeError("Optimisation did not converge or parameters are non-finite")
    if np.any(result.x[3:6] <= 0):
        raise RuntimeError("Non-positive scale factors")
    singular_values = np.linalg.svd(result.jac, compute_uv=False)
    tolerance = max(result.jac.shape) * np.finfo(float).eps * singular_values[0]
    if np.count_nonzero(singular_values > tolerance) != len(PARAMETERS):
        raise RuntimeError("Jacobian is not full rank")
    return result.x
def main():
    parameters = estimate_parameters(load_means(DATASET))
    output = dict(zip(PARAMETERS, map(float, parameters)))
    OUTPUT.parent.mkdir(parents=True, exist_ok=True)
    OUTPUT.write_text(json.dumps(output, indent=4) + "\n", encoding="utf-8")
if __name__ == "__main__":
    main()
```

Appendix C

Neural Network Training Code

The listing provides Python code for training the six architectures and the extended 3–10–3 run. Training and validation data retain the first 500 samples from each C pose and all 500 samples from additional poses, keeping every pose within one subset. The configuration file holds standardisation coefficients calculated from training data only. The program saves minimum-validation-MSE checkpoints, histories and comparison metrics. Model selection precedes the extended run and does not use test data. Recorded losses are dimensionless, while comparison MAE and RMSE are converted to m/s^2.

```
import os
os.environ.setdefault("TF_CPP_MIN_LOG_LEVEL", "2")
import json
from pathlib import Path
import numpy as np
import pandas as pd
import tensorflow as tf
ROOT = Path(__file__).resolve().parent
INPUTS = ["raw_x", "raw_y", "raw_z"]
TARGETS = ["cal_x", "cal_y", "cal_z"]
def load_data():
    cfg = json.loads((ROOT / "Config/preprocessing_config.json").read_text())
    mean = np.array([cfg["mean"][c] for c in INPUTS], np.float32)
    std = np.array([cfg["std"][c] for c in INPUTS], np.float32)
    pairs = []
    for name in ["train", "validation"]:
        data = pd.read_csv(ROOT / "Dataset" / f"{name}.csv")
        pairs.append(
            (
                (data[INPUTS].to_numpy(np.float32) - mean) / std,
                data[TARGETS].to_numpy(np.float32),
            )
        )
    return pairs
def build_model(hidden):
    tf.keras.utils.set_random_seed(42)
    tf.config.experimental.enable_op_determinism()
    model = tf.keras.Sequential(name="tinyml_calibration_mlp")
    model.add(tf.keras.layers.Input(shape=(3,), name="raw_acceleration"))
    for i, width in enumerate(hidden, start=1):
        model.add(
            tf.keras.layers.Dense(width, activation="tanh", name=f"hidden_{i}_tanh")
        )
    model.add(tf.keras.layers.Dense(3, name="calibrated_acceleration"))
    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
        loss="mse",
        metrics=[tf.keras.metrics.MeanAbsoluteError(name="mae")],
    )
    return model
class Monitor(tf.keras.callbacks.Callback):
```

```
    def __init__(self, x, y):
        super().__init__()
        self.x, self.y = x, y
    def on_epoch_end(self, epoch, logs=None):
        pred = self.model(self.x, training=False).numpy()
        logs["train_end_mse"] = float(np.mean((pred.astype(np.float64) - self.y) ** 2))
        logs["lr"] = float(
            tf.keras.backend.get_value(self.model.optimizer.learning_rate)
        )
def train(name, hidden, epochs, early_stopping=True):
    (x, y), (xv, yv) = load_data()
    model = build_model(hidden)
    folder = ROOT / "Runs" / name
    folder.mkdir(parents=True, exist_ok=True)
    callbacks = [Monitor(x, y)]
    if early_stopping:
        callbacks.append(
            tf.keras.callbacks.EarlyStopping(
                monitor="val_loss", patience=30, restore_best_weights=True
            )
        )
    callbacks += [
        tf.keras.callbacks.ModelCheckpoint(
            folder / "best.keras", monitor="val_loss", save_best_only=True
        ),
        tf.keras.callbacks.CSVLogger(folder / "history.csv"),
    ]
    history = model.fit(
        x,
        y,
        validation_data=(xv, yv),
        epochs=epochs,
        batch_size=256,
        shuffle=True,
        callbacks=callbacks,
        verbose=0,
    )
    model.save(folder / "last.keras")
    best = tf.keras.models.load_model(folder / "best.keras")
    error = (best(xv, training=False).numpy().astype(np.float64) - yv) * 9.80665
    row = dict(
        model=name,
        parameters=best.count_params(),
        epochs_run=len(history.history["loss"]),
        best_epoch=int(np.argmin(history.history["val_loss"]) + 1),
        best_val_mse=float(min(history.history["val_loss"])),
        val_mae=float(np.abs(error).mean()),
        val_rmse=float(np.sqrt((error**2).mean())),
    )
    (folder / "summary.json").write_text(json.dumps(row, indent=2))
    return row
def main():
    tf.config.threading.set_intra_op_parallelism_threads(1)
    tf.config.threading.set_inter_op_parallelism_threads(1)
    architectures = {
        "MLP_5": [5],
        "MLP_10": [10],
        "MLP_10_5": [10, 5],
        "MLP_3_3": [3, 3],
        "MLP_5_5": [5, 5],
        "MLP_8_5": [8, 5],
    }
    rows = []
    for name, hidden in architectures.items():
        rows.append(train(name, hidden, 500))
    results = pd.DataFrame(rows).sort_values(["best_val_mse", "parameters"])
```

```
108     results.to_csv(ROOT / "comparison.csv", index=False)
109     selected = results.iloc[0].to_dict()
110     (ROOT / "selected.json").write_text(json.dumps(selected, indent=2))
111     train("MLP_10_LONGRUN", [10], 2000, False)
112 if __name__ == "__main__":
113     main()
```

Appendix D

Network Pretraining Code for Continual Learning

The listing provides Python code for residual network pretraining in Section 7.2. It uses C07–C26 for training and static poses 41–50 for validation, saving the weights selected by the loss combining the norm constraint and directional regularisation. Network inputs and outputs are dimensionless; outputs are converted to $\mathrm{m/s^2}$ by multiplication by g_n.

```
from pathlib import Path
import random
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.utils.data import DataLoader, TensorDataset
ROOT = Path(__file__).resolve().parent.parent
OUTPUT = ROOT / "Models" / "cl_pretrained_best.pt"
SEED = 42
SCALE = 6000.0
EPOCHS = 1200
BATCH = 256
LR = 1e-3
PATIENCE = 100
LAMBDA_MAX = 0.1
RAMP = 200
def load_poses(folder, names):
    arrays = []
    for name in names:
        data = pd.read_csv(ROOT / "Dataset" / folder / f"{name}.csv")
        values = data[["raw_x", "raw_y", "raw_z"]].to_numpy(np.float32)
        values = values / SCALE
        if not np.isfinite(values).all():
            raise ValueError(f"Invalid data in {name}")
        arrays.append(values)
    return torch.tensor(np.concatenate(arrays))
class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.f = nn.Sequential(
            nn.Linear(3, 10), nn.Tanh(), nn.Linear(10, 3)
        )
        nn.init.zeros_(self.f[-1].weight)
        nn.init.zeros_(self.f[-1].bias)
    def forward(self, x):
        return x + self.f(x)
def loss(y, x, weight):
    yn = torch.linalg.vector_norm(y, dim=1).clamp_min(1e-8)
    xn = torch.linalg.vector_norm(x, dim=1).clamp_min(1e-8)
    norm_error = ((yn - 1) ** 2).mean()
    cosine = (y * x).sum(1) / (yn * xn)
```

```
    direction_error = (1 - torch.clamp(cosine, -1, 1)).mean()
    return norm_error + weight * direction_error
def main():
    random.seed(SEED)
    np.random.seed(SEED)
    torch.manual_seed(SEED)
    train = load_poses("C_Poses", [f"C{i:02d}" for i in range(7, 27)])
    validation = load_poses(
        "Static_Poses", [f"static_{i:03d}" for i in range(41, 51)]
    )
    model = Net()
    optimizer = torch.optim.Adam(model.parameters(), lr=LR)
    loader = DataLoader(
        TensorDataset(train), batch_size=BATCH, shuffle=True,
        generator=torch.Generator().manual_seed(SEED)
    )
    OUTPUT.parent.mkdir(parents=True, exist_ok=True)
    best = float("inf")
    wait = 0
    for epoch in range(1, EPOCHS + 1):
        weight = LAMBDA_MAX * min(1.0, epoch / RAMP)
        model.train()
        for (x,) in loader:
            optimizer.zero_grad()
            error = loss(model(x), x, weight)
            error.backward()
            optimizer.step()
        model.eval()
        with torch.no_grad():
            value = loss(model(validation), validation, weight).item()
        if value < best - 1e-12:
            best = value
            wait = 0
            torch.save(model.state_dict(), OUTPUT)
        else:
            wait += 1
            if wait >= PATIENCE:
                break
if __name__ == "__main__":
    main()
```

Appendix E

Code for Adaptation to Calibration Drift

The code implements the update procedure in Section 7.4. Function `adapt` takes the pretrained model and altered, normalised poses in `float32` format, returning the updated model and norm MAE before and after each pose. MAE is dimensionless and is subsequently multiplied by g_n to express it in m/s^2.

```
import numpy as np
import torch
from torch import nn
class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.f = nn.Sequential(
            nn.Linear(3, 10), nn.Tanh(), nn.Linear(10, 3)
        )
    def forward(self, x):
        return x + self.f(x)
def predict(model, x):
    with torch.no_grad():
        y = model(torch.as_tensor(x, dtype=torch.float32))
    return y.numpy().astype(np.float64)
def norm_mae(model, x):
    return float(np.abs(np.linalg.norm(predict(model, x), axis=1)-1).mean())
def adapt(model, poses, seed=42, lr=1e-3, updates=100,
          current_count=128, replay_count=128, retained=100):
    rng = np.random.default_rng(seed)
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    replay = []
    for step, (name, current) in enumerate(poses, start=1):
        before = norm_mae(model, current)
        previous = np.concatenate(replay) if replay else current
        model.train()
        for _ in range(updates):
            new = current[rng.integers(len(current), size=current_count)]
            old = previous[rng.integers(len(previous), size=replay_count)]
            batch = torch.from_numpy(np.concatenate([new, old]))
            optimizer.zero_grad()
            norms = torch.linalg.vector_norm(model(batch), dim=1)
            loss = (norms-1).square().mean()
            loss.backward()
            optimizer.step()
        model.eval()
        chosen = rng.choice(len(current), retained, replace=False)
        replay.append(current[chosen].copy())
        yield step, name, model, before, norm_mae(model, current)
```

References

[1] Simon Desrochers. *Optimum Design of Simplicial Uniaxial Accelerometers*. Master of Engineering thesis, McGill University, 2009. Figure 1.1, p. 2.

McGill University – PDF document.

Accessed: September 2026.

[2] Harvey Weinberg. *Dual Axis Low g Fully Integrated Accelerometers*. Analog Dialogue, Analog Devices. Figure 1 and section *Sensor Structure*.

Analog Devices – article.

Accessed: September 2026.

[3] Analog Devices. *ADXL330: Small, Low Power, 3-Axis Accelerometer*. Datasheet, revision A, section *Theory of Operation*.

Analog Devices – datasheet ADXL330.

Accessed: September 2026.

[4] Martín Monteiro, Cecilia Cabeza and Arturo C. Martí. *Acceleration Measurements Using Smartphone Sensors: Dealing with the Equivalence Principle*. arXiv:1406.3867, 2014.

arXiv – article.

Accessed: September 2026.

[5] Christopher J. Fisher. *Using an Accelerometer for Inclination Sensing*. Application note AN-1057, Analog Devices.

Analog Devices – application note AN-1057.

Accessed: September 2026.

[6] National Bureau of Standards. *Gravity Measurements and the Standards Laboratory*. Technical Note 491.

NIST – PDF document, Technical Note 491.

Accessed: September 2026.

[7] Analog Devices. *Accelerometer Specifications – Quick Definitions*. Definitions of key accelerometer specifications.

Analog Devices – accelerometer specifications.

Accessed: September 2026.

[8] Mark Pedley. *High-Precision Calibration of a Three-Axis Accelerometer*. Application note AN4399, revision 2.0, Freescale Semiconductor, October 2015.

NXP – application note AN4399.

Accessed: September 2026.

[9] Analog Devices. *Best Engineering Practices When Using ADXL Accelerometers*. Technical guide, Analog Devices Wiki.

Analog Devices – guide to using ADXL accelerometers.

Accessed: September 2026.

[10] Aston Zhang, Zachary C. Lipton, Mu Li and Alexander J. Smola. *Dive into Deep Learning*. Cambridge University Press, 2023. Online edition, Sections 3.6, 5.1, 5.3 and 12.3; Figure 5.1.1.

Dive into Deep Learning – online book.

Accessed: September 2026.

[11] STMicroelectronics. *Getting Started with X-CUBE-AI Expansion Package for Artificial Intelligence (AI)*. User manual UM2526.

STMicroelectronics – user manual UM2526.

Accessed: September 2026.

[12] Murata Electronics Oy. *SCL3300-D01: 3-axis Inclinometer with Angle Output and Digital SPI Interface*. Datasheet, document 4921, revision 4. Sections 2.2, 2.3, 3 and 6.

Murata – datasheet SCL3300-D01.

Accessed: September 2026.

[13] Murata Manufacturing Co., Ltd. *Sensor Boards for Digital Accelerometers/Inclinometer: SCL3300 Series PCB*. SCL3300-D01-PCB module page and image.

Murata – SCL3300-D01-PCB module.

Accessed: September 2026.

[14] STMicroelectronics. *STM32H743ZI*. Microcontroller specification documentation.

STMicroelectronics – STM32H743ZI microcontroller.

Accessed: September 2026.

[15] STMicroelectronics. *STM32H7 Nucleo-144 Boards (MB1364)*. User manual UM2407, revision 6, April 2026.

STMicroelectronics – Nucleo MB1364 board manual.

Accessed: September 2026.

[16] Farnell. *STMicroelectronics NUCLEO-H743ZI2*. Catalogue page, product code 3106010. Board photograph, front and rear views.

Farnell – NUCLEO-H743ZI2 photograph.

Accessed: September 2026.

[17] Massimo Duchi and Edoardo Ida'. *Total Least Squares In-Field Identification for MEMS-Based Inertial Measurement Units*. Robotics, vol. 13, no. 11, article 156, 2024. Section 3.

Robotics – article by Duchi and Ida'.

Accessed: September 2026.

[18] David Tedaldi, Alberto Pretto and Emanuele Menegatti. *A Robust and Easy to Implement Method for IMU Calibration without External Equipments*. Proceedings of the IEEE International Conference on Robotics and Automation (ICRA), pp. 3042–3049, 2014.

IEEE – article by Tedaldi, Pretto and Menegatti.

Accessed: September 2026.

[19] SciPy. *scipy.optimize.least_squares*. Official documentation, version 1.18.0.

SciPy – least_squares function documentation.

Accessed: September 2026.

[20] Diederik P. Kingma and Jimmy Ba. *Adam: A Method for Stochastic Optimization*. International Conference on Learning Representations (ICLR), 2015.

arXiv – article by Kingma and Ba.

Accessed: September 2026.

[21] Keras. *EarlyStopping*. Official Keras API documentation.

Keras – EarlyStopping documentation.

Accessed: September 2026.

[22] Scikit-learn developers. *Neural network models (supervised)*. Official documentation, Tips on Practical Use section.

Scikit-learn – practical guidance for MLP networks.

Accessed: September 2026.

[23] Scikit-learn developers. *StandardScaler*. Official API documentation.

Scikit-learn – input standardisation.

Accessed: September 2026.

[24] Keras. *Adam*. Official Keras API documentation.

Keras – Adam optimiser configuration.

Accessed: September 2026.

[25] Keras contributors. *Keras FAQ: Why is my training loss much higher than my testing loss?* Official documentation on loss computation during training and evaluation.

Keras – training and validation loss computation.

Accessed: September 2026.

[26] ONNX contributors. *tf2onnx: Convert TensorFlow, Keras, Tensorflow.js and Tflite models to ONNX*. Official repository, Getting started and CLI reference sections.

ONNX – tf2onnx converter documentation.

Accessed: September 2026.

[27] STMicroelectronics. *ST Edge AI Core – Command-line Interface*. Official documentation, Generate command.

STMicroelectronics – inference code generation.

Accessed: September 2026.

[28] STMicroelectronics. *Embedded Inference Client ST Edge AI API (st-ai)*. Official embedded inference interface documentation.

STMicroelectronics – st-ai inference interface.

Accessed: September 2026.

[29] STMicroelectronics. *STM32Cube AI Studio documentation*. Official documentation, version 1.3.0, features and development procedure.

STMicroelectronics – STM32Cube AI Studio documentation.

Accessed: September 2026.

[30] Google. *LiteRT for Microcontrollers*. Google AI Edge documentation, overview and deployment procedure.

Google – microcontroller inference.

Accessed: September 2026.

[31] Arm. *CMSIS-NN: CMSIS NN Software Library*. Official library documentation.

Arm – CMSIS-NN library.

Accessed: September 2026.

[32] Edge Impulse. *C++ SDK*. Official processing and inference software documentation.

Edge Impulse – C++ inference SDK.

Accessed: September 2026.

[33] Edge Impulse. *Run Cube.MX CMSIS-Pack*. Official STM32CubeMX and STM32CubeIDE integration documentation.

Edge Impulse – deployment through CMSIS-Pack for STM32.

Accessed: September 2026.

[34] Timothée Lesort, Vincenzo Lomonaco, Andrei Stoian, Davide Maltoni, David Filliat and Natalia Díaz-Rodríguez. *Continual learning for robotics: Definition, framework, learning strategies, opportunities and challenges*. Information Fusion, vol. 58, pp. 52–68, 2020.

Elsevier – article on continual learning in robotics.

Accessed: September 2026.

[35] Liyuan Wang, Xingxing Zhang, Hang Su and Jun Zhu. *A Comprehensive Survey of Continual Learning: Theory, Method and Application*. IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 46, no. 8, pp. 5362–5383, 2024.

IEEE – survey of continual learning theory, methods and applications.

Accessed: September 2026.

[36] James Kirkpatrick et al. *Overcoming catastrophic forgetting in neural networks*. Proceedings of the National Academy of Sciences, vol. 114, no. 13, pp. 3521–3526, 2017.

PNAS – article on knowledge retention in neural networks.

Accessed: September 2026.

[37] Vincenzo Lomonaco and Davide Maltoni. *CORe50: a New Dataset and Benchmark for Continuous Object Recognition*. Proceedings of the 1st Annual Conference on Robot Learning, Proceedings of Machine Learning Research, vol. 78, pp. 17–26, 2017.

PMLR – CORe50 dataset and protocol.

Accessed: September 2026.

[38] Jipu Li, Ruyi Huang, Zhuyun Chen, Guolin He, Konstantinos C. Gryllias and Weihua Li. *Deep continual transfer learning with dynamic weight aggregation for fault diagnosis of industrial streaming data under varying working conditions*. Advanced Engineering Informatics, vol. 55, article 101883, 2023.

Elsevier – continual learning for fault diagnosis.

Accessed: September 2026.

[39] Ji Lin, Ligeng Zhu, Wei-Ming Chen, Wei-Chen Wang, Chuang Gan and Song Han. *On-Device Training Under 256KB Memory*. Advances in Neural Information Processing Systems, vol. 35, 2022.

NeurIPS – training on memory-constrained devices.

Accessed: September 2026.

[40] Tyler L. Hayes and Christopher Kanan. *Online Continual Learning for Embedded Devices*. Proceedings of The 1st Conference on Lifelong Learning Agents, Proceedings of Machine Learning Research, vol. 199, pp. 744–766, 2022.

PMLR – continual learning on embedded devices.

Accessed: September 2026.

[41] PyTorch contributors. *Linear*. Official documentation for module `torch.nn.Linear`.

PyTorch – affine neural network layers.

Accessed: September 2026.

[42] Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun. *Deep Residual Learning for Image Recognition*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 770–778, 2016.

IEEE – article on residual neural network formulation.

Accessed: September 2026.